Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y Tecnología

Máster en Computación Cuántica

La Evolución de la Ciencia de Datos hacia la Computación Cuántica: Análisis de la Ventaja Cuántica en Casos Aplicados

Trabajo Fin de Máster

presentado por: Juan David Prada Malagón

Dirigido por: Ricardo Rafael Domínguez Reyes

Ciudad: Barcelona, España

Fecha: November 2, 2026

Índice general

Índice de figuras iii

Índice de cuadros v

Resumen vii

Abstract viii

1. Introducción 1

1.1. Motivación y justificación del tema 1

1.2. Planteamiento del problema 1

1.3. Estructura del trabajo 2

2. Contexto y estado de la técnica 4

2.1. Antecedentes históricos 4

2.2. Computación cuántica en la era NISQ 5

2.3. Aprendizaje automático cuántico 6

2.4. Optimización cuántica aplicada 7

2.5. Análisis crítico de los trabajos previos 8

3. Objetivos 9

3.1. Objetivo general 9

3.2. Objetivos específicos 9

4. Metodología y desarrollo experimental 10

4.1. Entorno tecnológico 10

4.2. Diseño experimental 11

4.3. Clasificación 11

4.3.1. Datasets 11

4.3.2. Preparación compartida 15

4.3.3. Representación en el espacio de Hilbert 15

4.3.4. Modelos 17

4.3.5. Métricas de clasificación y latencia 17

4.4. Optimización 18
4.4.1. Problemas e instancias 18
4.4.2. Preparación: de problema a QUBO/Ising 20
4.4.3. Solvers 21
4.4.4. Métricas de optimización y latencia 22
4.5. Comparación clásico vs cuántico 22
4.6. Reproducibilidad 22
4.7. Limitaciones del diseño 23

5. Resultados y análisis **25**
5.1. Clasificación 25
5.1.1. Iris 26
5.1.2. Bars and Stripes 28
5.1.3. Ising 30
5.1.4. Lectura de la familia 31
5.2. Optimización 32
5.2.1. TSP 34
5.2.2. Minimum dominating set 35
5.2.3. MaxCut 36
5.2.4. Lectura de la familia 37
5.3. Discusión: la representación como eje 38

6. Conclusiones **40**
6.1. Conclusiones 40

7. Líneas futuras de investigación **42**

Bibliografía **43**

A. Apéndices **51**
A.1. Repositorio del proyecto 51
A.2. Catálogo de notebooks 51
A.3. Formato de las salidas 53
A.4. Reproducción 53

Índice de figuras

1.1. Estructura del TFM. 3

2.1. Evolución del hardware cuántico en la era NISQ (2016–2024) 5

4.1. Pipeline experimental común: dato → preparación → modelo/solver → métricas agregadas. 12

4.2. Iris antes de los clasificadores. Proyección visual de las cuatro variables originales; el modelo no usa esta PCA. Círculos huecos: test del protocolo. . 13

4.3. Bars and Stripes antes de aplanar y antes de cualquier clasificador. Cada panel es un patrón 4×4; $y \in \{+1, -1\}$ es la clase del catálogo. 13

4.4. Bars and Stripes tras la reducción a componentes y antes de cualquier clasificador. Las dos clases comparten el mismo plano. Círculos huecos: test. . 14

4.5. Sistema de Ising: cadena 1D de cuatro espines con acoplamiento J entre vecinos y campo transversal h, en layout abierto o cerrado. Las features son las amplitudes del estado fundamental de este Hamiltoniano para cada h. . 14

4.6. Preparación compartida y bifurcación C/Q. Tras split, PCA (si $d > 4$) y MinMax$[0, \pi]$, el mismo X alimenta baselines clásicos; en la rama cuántica pasa antes por el embedding al espacio de Hilbert. Ambas ramas se evalúan con las mismas métricas de calidad y latencia. 15

4.7. Instancia de TSP antes de cualquier solver. Cinco nodos, ocho aristas y sus distancias. 19

4.8. Instancia de MDS antes de cualquier solver. Seis vértices y seis aristas. . . . 19

4.9. Instancia de MaxCut antes de cualquier solver. Seis vértices y doce aristas; el corte óptimo es 8. 20

4.10. Optimización: la representación QUBO/Ising es compartida por SA, QAOA y VQE; BF/greedy/ILP operan sobre la formulación nativa o equivalente. . 21

5.1. Accuracy de test media de la familia clásica (SVM, Random Forest, XGBoost) y de la cuántica (QSVM, QNN, VQC) en cada dataset. $\Delta(Q - C)$ es la diferencia entre ambas medias. 25

5.2. Fronteras de decisión en Iris. Cada panel evalúa el modelo entrenado sobre el plano de las dos primeras componentes principales de su entrada, con las demás direcciones fijas en la media del train. Fila superior: SVM, Random Forest y XGBoost; fila inferior: QSVM, QNN y VQC. Puntos: test; círculos: errores de test. 27
5.3. Fronteras de decisión en Bars and Stripes, con la misma construcción que la Figura 5.2. Las dos primeras componentes no separan las clases, así que el plano solo muestra una sección de cada frontera. Fila superior: SVM, Random Forest y XGBoost; fila inferior: QSVM, QNN y VQC. 29
5.4. Accuracy de test por modelo en cada dataset y su media. 31
5.5. F1 de test medio sobre los tres datasets frente al tiempo medio de entrenamiento (escala logarítmica). 32
5.6. Ruta óptima del TSP restringido, de coste 16. 34
5.7. MDS. Izquierda: conjunto dominante mínimo, {Ana, Enrique}, devuelto por fuerza bruta, ILP, SA, VQE y siete de las ocho semillas de QAOA. Derecha: solución del greedy, de tamaño 3. 35
5.8. Un corte óptimo de MaxCut (valor 8). El óptimo es degenerado: 18 asignaciones, 9 cortes distintos salvo complemento. 37
5.9. Success rate medio frente al tiempo medio por solver, promediado sobre TSP, MDS y MaxCut. 38

A.1. Árbol de carpetas del repositorio. Se omiten las figuras y los PDF compilados. 52

Índice de cuadros

4.1. Entorno software del estudio, con versiones en el orden en que aparecen los paquetes. Python es la versión del entorno virtual de ejecución; las demás versiones son las fijadas en las dependencias del repositorio, que no fijan `qiskit-machine-learning`. 10

4.2. Ejes del estudio. La representación no es una tercera familia de datos: es la transformación que comparten clasificación y optimización. 11

4.3. Datasets de clasificación. Partición estratificada con 40 % de test y semilla 123. 12

4.4. Modelos de clasificación. La configuración es la misma en los tres datasets. . 17

4.5. Métricas de calidad y latencia en clasificación. 18

4.6. Instancias de optimización. 18

4.7. Configuración de los solvers por problema. Fuerza bruta, greedy e ILP son deterministas y se ejecutan una vez; SA usa enfriamiento geométrico de razón 0.995. 21

4.8. Métricas de calidad y latencia en optimización. C es el coste o el corte obtenido; $C^\star$ es el óptimo de la instancia. 22

4.9. Ciclo de modelado comparado. La columna cualitativa delimita qué es común y qué es propio de cada familia; la cuantitativa aplica las métricas de calidad y latencia. 23

5.1. Iris, conjunto de test. Gap $= \mathrm{Acc}_{\mathrm{train}} - \mathrm{Acc}_{\mathrm{test}}$. El VQC no produce scores y no tiene AUC. 26

5.2. Bars and Stripes, conjunto de test (480 muestras). 28

5.3. Ising, conjunto de test (80 muestras). El SVM y el QSVM fallan la misma cantidad de muestras (una). 30

5.4. Resultados de optimización. $\bar{C}$ es el coste medio sobre ejecuciones (corte medio en MaxCut) y t, el tiempo medio por ejecución. Evaluaciones de energía por ejecución (TSP, MDS y MaxCut): SA 5001, 4001 y 4000; QAOA 356, 4000 y 40 pasos de Adagrad; VQE 720, 600 y 240. En VQE y en QAOA sobre MDS coinciden con el máximo permitido, reinicios $\times$ iteraciones de COBYLA. 33

5.5. Cada instancia como espacio de 2^6 bitstrings. En el TSP solo son factibles los bitstrings que seleccionan una única ruta y que esa ruta no use aristas ausentes, lo que deja 2 de las 6 rutas candidatas. En MaxCut los 18 bitstrings óptimos forman 9 cortes distintos salvo complemento. 33

7.1. Líneas futuras de investigación. 42

A.1. Primer nivel del repositorio. 51
A.2. Organización del corpus experimental. 53
A.3. Ficheros de salida del corpus experimental. 54
A.4. Pasos para reproducir los resultados, el documento y la defensa. 54

Resumen

Este Trabajo Fin de Máster (TFM) compara algoritmos de computación cuántica, algoritmos cuántico-inspirados y métodos clásicos en dos áreas de la ciencia de datos, clasificación y optimización combinatoria, con un mismo protocolo de preparación y medida y en simulación de 4 a 6 qubits.

En clasificación se usan tres conjuntos de datos: Iris, Bars and Stripes e Ising unidimensional (clasificación de fases). Se comparan QSVM, QNN y VQC frente a SVM, Random Forest y XGBoost, con el mismo protocolo de partición, escalado y, cuando hace falta, PCA.

En optimización se resuelven tres problemas: el Problema del Viajante (TSP), el Minimum Dominating Set (MDS) y Maximum Cut sobre una instancia de HamLib. Se evalúan QAOA y VQE frente a fuerza bruta, greedy, ILP y Simulated Annealing sobre QUBO.

El SVM obtiene el mejor F1 o empata con el mejor en los tres datasets y es el más rápido de entrenar. Los modelos cuánticos quedan por debajo en Iris y Bars and Stripes; el QSVM empata con el SVM solo en Ising, el dataset en el que los seis modelos alcanzan al menos 0.95 de accuracy. En optimización, todos los métodos salvo el greedy en MDS alcanzan el óptimo o se quedan cerca, pero QAOA y VQE tardan de 0.6 a 12.6 segundos donde la enumeración tarda menos de un milisegundo, y QAOA falla una de ocho semillas en MDS. En este régimen no hay ventaja cuántica: la paridad de calidad, cuando aparece, llega con un coste temporal mayor.

Palabras clave: computación cuántica, quantum-inspired, ciencia de datos, clasificación, optimización combinatoria, Iris, Bars and Stripes, Ising, TSP, MDS, MaxCut, QSVM, VQC, QAOA, VQE, NISQ.

Abstract

This Master's Thesis compares quantum computing algorithms, quantum-inspired methods and classical baselines on two areas of data science, classification and combinatorial optimisation, under a single preparation and measurement protocol and in simulation with 4 to 6 qubits.

Classification uses three datasets: Iris, Bars and Stripes and one-dimensional Ising (phase classification). QSVM, QNN and VQC are compared with SVM, Random Forest and XGBoost under the same split, scaling and, when needed, PCA protocol.

Optimisation covers three problems: the Travelling Salesperson Problem (TSP), the Minimum Dominating Set (MDS) and Maximum Cut on a HamLib instance. QAOA and VQE are evaluated against brute force, greedy, ILP and Simulated Annealing on QUBO.

The SVM obtains the best or tied-best F1 on all three datasets and is the fastest to train. Quantum models fall behind on Iris and Bars and Stripes; the QSVM ties the SVM only on Ising, the dataset on which all six models reach at least 0.95 accuracy. In optimisation every method except greedy on MDS reaches or nearly reaches the optimum, but QAOA and VQE take 0.6 to 12.6 seconds where enumeration takes under a millisecond, and QAOA misses one of eight seeds on MDS. There is no quantum advantage in this regime: quality parity, where it appears, comes at a higher time cost.

Keywords: quantum computing, quantum-inspired, data science, classification, combinatorial optimisation, Iris, Bars and Stripes, Ising, TSP, MDS, MaxCut, QSVM, VQC, QAOA, VQE, NISQ.

1. Introducción

La computación cuántica y la ciencia de datos convergen en aprendizaje automático cuántico (QML) y optimización cuántico-inspirada. En la era NISQ, la ventaja práctica frente a métodos clásicos sigue sin estar demostrada de forma sistemática (Preskill, 2018; Biamonte et al., 2017). Los algoritmos variacionales (QAOA, VQE, VQC) y los kernels cuánticos son el enfoque habitual en hardware intermedio (Farhi et al., 2014; Cerezo et al., 2021; Havlíček et al., 2019); en demos pequeñas y con protocolo justo, los baselines clásicos suelen igualar o ganar (Bowles et al., 2024; Yu et al., 2026; Z-537-437, n.d.). El interés estratégico por la tecnología, visible en su consideración como asunto de seguridad nacional (Quantum Zeitgeist, n.d.c), hace más necesario separar lo medido de lo prometido.

Este TFM hace una comparación reproducible en **simulación clásica**: tres tareas de clasificación y tres de optimización combinatoria, con el mismo preproceso y con métricas de calidad y de tiempo. El marco usa tres ejes: *qué* (clasificar), *cómo* (optimizar) y *representación* (feature map / QUBO–Ising) (Lloyd et al., 2020; PennyLane, n.d.c; Parzygnat et al., 2025).

1.1 Motivación y justificación del tema

La teoría apunta aceleraciones en casos concretos (Grover, 1996; Shor, 1994) y, en QML, a kernels en espacios de características grandes (Havlíček et al., 2019; Schuld & Killoran, 2019). En la práctica, la ventaja de un kernel cuántico depende de la estructura del dato (Huang et al., 2021) y queda limitada por pocos qubits, ruido y concentración del kernel (Preskill, 2018; Thanasilp et al., 2024; McClean et al., 2018). Hace falta medir precisión y wall-clock con el mismo protocolo, no demos asimétricas (Bowles et al., 2024).

1.2 Planteamiento del problema

Pregunta: *¿En las instancias y el régimen NISQ simulados aquí, los enfoques cuánticos o cuántico-inspirados mejoran la calidad o el esfuerzo frente a baselines clásicos?*

1. **Clasificación.** Datasets: Iris (scikit-learn developers, n.d.), Bars and Stripes (PennyLane, n.d.a,n) e Ising 1D (fases) (PennyLane, n.d.h,n). Modelos: SVM, Random Forest, XGBoost frente a QSVM, QNN, VQC (PennyLane, n.d.m). Mismo split,

escalado MinMax$[0, \pi]$ y PCA a 4 cuando $d > 4$.

2. **Optimización.** Problemas: TSP, Minimum Dominating Set (MDS) y MaxCut (HamLib) (Sawaya et al., 2024; PennyLane, n.d.f,n). Solvers: fuerza bruta, greedy, ILP (Gurobi Optimization, n.d.), QUBO+SA frente a QAOA y VQE (Farhi et al., 2014; Peruzzo et al., 2014; Lubinski et al., 2024). Métricas: coste/corte, gap, éxito y tiempo.

3. **Representación.** Embedding / feature map en QML (PennyLane, n.d.d,n); QUBO–Ising en optimización (Lucas, 2014).

Iris y Bars and Stripes son datasets convencionales para QML; Ising es dato de sistema con etiqueta de fase (PennyLane, n.d.n,n). No hay datasets de empresa en este corpus. La aportación es el protocolo y la lectura de límites, no un claim de ventaja en este régimen (Bowles et al., 2024).

1.3 Estructura del trabajo

La Figura 1.1 agrupa los capítulos en marco, experimentos y cierre.

- **Capítulo 2 – Contexto y estado de la técnica:** era NISQ, QML, QAOA/VQE y limitaciones de los trabajos previos.
- **Capítulo 3 – Objetivos:** objetivo general y objetivos específicos.
- **Capítulo 4 – Metodología:** entorno, datasets e instancias, representación (feature maps y QUBO/Ising), modelos, solvers y métricas.
- **Capítulo 5 – Resultados y análisis:** resultados por familia, dataset e implementación, y discusión centrada en la representación.
- **Capítulo 6 – Conclusiones:** respuesta a la pregunta de investigación.
- **Capítulo 7 – Líneas futuras de investigación:** propuestas que extienden el estudio.



Figura 1.1: Estructura del TFM.

Fuente: elaboración propia.

2. Contexto y estado de la técnica

Este capítulo presenta los fundamentos conceptuales y el estado actual de la técnica en computación cuántica aplicada a la ciencia de datos. Se revisan los trabajos previos más relevantes, se identifican las limitaciones de los enfoques existentes y se sitúa la contribución de este trabajo en el contexto académico y profesional actual.

2.1 Antecedentes históricos

El origen formal de la computación cuántica como disciplina se remonta a los trabajos de Feynman (1982), quien propuso el uso de sistemas cuánticos para simular la naturaleza de forma eficiente, argumentando que un computador clásico no podía simular un sistema cuántico sin un coste exponencial. Poco después, Deutsch (1985) formalizó el concepto de computador cuántico universal y demostró la existencia de problemas que un computador cuántico podría resolver de forma más eficiente que cualquier computador clásico.

El interés por la computación cuántica se intensificó con los algoritmos de Shor (1994) y Grover (1996), que demostraron aceleraciones exponenciales y cuadráticas respectivamente frente a los mejores algoritmos clásicos conocidos para factorización de enteros y búsqueda no estructurada. El primero descansa en la transformada cuántica de Fourier y el segundo en la amplificación de amplitud (PennyLane, n.d.k,n). Estos resultados establecieron la viabilidad teórica de la ventaja cuántica y sentaron las bases para el desarrollo de hardware cuántico experimental. Los fundamentos formales del modelo de computación cuántica, incluyendo los postulados de la mecánica cuántica, las puertas cuánticas y los circuitos cuánticos, se encuentran desarrollados en detalle en Nielsen & Chuang (2010); CERN & Atos (2021) y Quantum Zeitgeist (n.d.b) ofrecen introducciones más aplicadas.

En paralelo, el campo del aprendizaje automático experimentó un crecimiento extraordinario impulsado por el aumento de la capacidad computacional y la disponibilidad de grandes volúmenes de datos (Jordan & Mitchell, 2015). La confluencia de ambas disciplinas dio lugar al campo del aprendizaje automático cuántico (*Quantum Machine Learning*, QML), cuyo objetivo es explorar si las capacidades de procesamiento cuántico pueden ofrecer ventajas en tareas de aprendizaje estadístico (Biamonte et al., 2017). La relación también se da en sentido inverso: el aprendizaje automático se usa para caracterizar estados y controlar dispositivos cuánticos (Fastovets et al., 2019).

2.2 Computación cuántica en la era NISQ

El estado actual del hardware cuántico se enmarca en lo que Preskill (2018) denominó la era NISQ (*Noisy Intermediate-Scale Quantum*), caracterizada por dispositivos con entre 50 y varios cientos de qubits físicos, afectados por errores de decoherencia y ruido cuántico, y sin capacidad para implementar corrección de errores cuánticos a gran escala. Esta realidad impone restricciones fundamentales sobre la profundidad de los circuitos ejecutables y la fiabilidad de los resultados obtenidos. La Figura 2.1 sitúa en el tiempo los procesadores de IBM y Google de esta etapa.

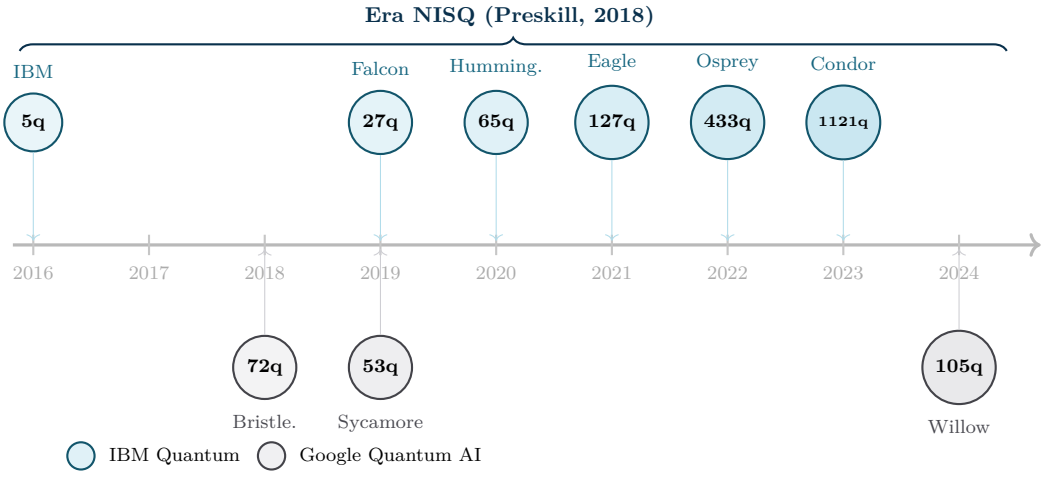


Figura 2.1: Evolución del hardware cuántico en la era NISQ (2016–2024). Los nodos superiores (azul) corresponden a procesadores IBM Quantum; los inferiores (violeta) a Google Quantum AI. Se indica el año de presentación de cada procesador; los recuentos de qubits de IBM proceden de sus anuncios públicos. Los números indican el recuento de qubits físicos disponibles. Arute et al. (2019) presentaron con Sycamore (2019) un experimento de muestreo de circuitos aleatorios que declararon fuera del alcance de la simulación clásica de entonces; Pan et al. (2022) lo replicaron después con algoritmos clásicos mejorados. Willow (2024) se asocia al trabajo de Google Quantum AI (2025) sobre corrección de errores por debajo del umbral del código de superficie.
Fuente: elaboración propia.

En respuesta a estas limitaciones, se han desarrollado los algoritmos variacionales cuánticos (*Variational Quantum Algorithms*, VQA), que combinan un circuito cuántico parametrizado con un optimizador clásico en un bucle híbrido (Cerezo et al., 2021; Quantum Zeitgeist, n.d.a). El diseño del ansatz condiciona su expresividad y su capacidad de en-

trelazamiento (Sim et al., 2019), y la recarga repetida de los datos en el circuito permite construir clasificadores universales con pocos qubits (Pérez-Salinas et al., 2020). Entre los más destacados se encuentran el Quantum Approximate Optimization Algorithm (QAOA) (Farhi et al., 2014), diseñado para problemas de optimización combinatoria, y el Variational Quantum Eigensolver (VQE) (Peruzzo et al., 2014), orientado a la simulación de sistemas moleculares. Los algoritmos variacionales son hoy la estrategia principal para buscar ventaja cuántica práctica en hardware NISQ (Cerezo et al., 2021).

No obstante, la comunidad científica mantiene un debate activo sobre si los algoritmos NISQ pueden alcanzar ventaja cuántica genuina frente a los mejores algoritmos clásicos. McClean et al. (2018) mostraron que, en circuitos parametrizados aleatorios, la probabilidad de que el gradiente sea distinto de cero con precisión fija es exponencialmente pequeña en el número de qubits (*barren plateaus*), lo que dificulta el entrenamiento de los circuitos variacionales a medida que el número de qubits crece. Cerezo et al. (2025) discuten además si los modelos libres de *barren plateaus* pueden simularse eficientemente de forma clásica, lo que cuestiona qué ventaja pueden ofrecer.

2.3 Aprendizaje automático cuántico

Biamonte et al. (2017) revisaron el aprendizaje automático cuántico distinguiendo si la máquina que aprende es clásica o cuántica y si los datos son clásicos o estados cuánticos producidos por sensores o experimentos. Para el contexto de este trabajo, el caso más relevante es el de datos clásicos codificados como estados cuánticos y procesados por un circuito.

Los métodos basados en kernels cuánticos (*Quantum Kernel Methods*) han recibido especial atención. Schuld & Killoran (2019) propusieron interpretar los circuitos cuánticos como mapas de características (*feature maps*) hacia espacios de Hilbert de alta dimensión, y Havlíček et al. (2019) implementaron experimentalmente el estimador de kernel cuántico en un procesador superconductor, argumentando que ciertos mapas son difíciles de simular clásicamente. Schuld (2021) mostró después que los modelos cuánticos supervisados basados en circuitos parametrizados son, formalmente, métodos de kernel. Sin embargo, Huang et al. (2021) establecieron condiciones teóricas precisas bajo las cuales los kernels cuánticos pueden ofrecer una ventaja de aprendizaje sobre los kernels clásicos, concluyendo que dicha ventaja depende críticamente de la estructura geométrica de los datos de

entrada. Liu et al. (2021) probaron una aceleración rigurosa en aprendizaje supervisado, pero sobre un problema construido a partir del logaritmo discreto; el grado de acuerdo entre un kernel y las etiquetas puede medirse con el alineamiento centrado de Cortes et al. (2012).

En el contexto de la clasificación supervisada con datos reales, Haug et al. (2023) propusieron el uso de medidas aleatorizadas para escalar los métodos de kernel cuántico a conjuntos de datos de gran tamaño, abordando uno de los principales cuellos de botella prácticos de estos métodos: el coste de estimar el kernel crece cuadráticamente con el tamaño del conjunto de datos. Su método reduce el tiempo de cómputo cuántico a escala lineal con dicho tamaño, aunque en general escala de forma exponencial con el número de qubits, y lo demostraron clasificando imágenes en un procesador cuántico de IBM.

2.4 Optimización cuántica aplicada

El QAOA, propuesto originalmente por Farhi et al. (2014), es un algoritmo híbrido cuántico-clásico que aproxima la solución de problemas de optimización combinatoria formulados como problemas de optimización cuadrática binaria sin restricciones (QUBO). Muchos de esos problemas, como el TSP o el conjunto dominante mínimo, son NP-difíciles (Garey & Johnson, 1979), y el annealing simulado, uno de sus heurísticos clásicos habituales, tiene una versión cuántica con aceleración cuadrática respecto del gap espectral del proceso clásico (Somma et al., 2008). Lucas (2014) recoge formulaciones de Ising para muchos problemas NP de interés práctico, entre ellos el TSP, la cobertura de vértices y la partición de grafos.

Zhou et al. (2020) estudiaron numéricamente el QAOA sobre MaxCut en grafos regulares, hasta $p \leq 50$ y $N \leq 22$ vértices, y propusieron estrategias heurísticas de inicialización de parámetros cuyo coste crece de forma polinómica en p; observaron además que la calidad de la solución mejora al aumentar la profundidad. No obstante, la ventaja del QAOA frente a los mejores heurísticos clásicos sigue siendo una cuestión abierta en instancias de gran escala (Lubinski et al., 2024).

Desde una perspectiva aplicada, Vikstål et al. (2020) simularon el QAOA sobre instancias del problema de asignación de aeronaves a vuelos (*tail-assignment*) derivadas de datos reales de una aerolínea, reducidas a 8, 15 y 25 qubits. Encontraron que repetidas ejecuciones identifican la solución factible con probabilidad cercana a uno, que los paráme-

tros variacionales siguen patrones que permiten interpolarlos entre profundidades, y que la probabilidad de éxito por medición depende de la conectividad del grafo del problema. Estos resultados subrayan la importancia de realizar evaluaciones experimentales sobre datos reales más allá de las pruebas de concepto teóricas.

2.5 Análisis crítico de los trabajos previos

La revisión de la literatura permite identificar limitaciones recurrentes en los estudios existentes que este trabajo aborda:

- **Comparaciones asimétricas**: muchos trabajos evalúan el modelo cuántico de forma aislada o frente a baselines clásicos poco ajustados, con preprocesos distintos para cada rama (Bowles et al., 2024). Los benchmarks sistemáticos con el mismo protocolo muestran que los modelos clásicos de referencia suelen igualar o superar a los cuánticos en datasets pequeños (Bowles et al., 2024; XanaduAI, n.d.; Wang, n.d.).
- **Afirmaciones de ventaja sin control**: en la divulgación circulan resultados de superioridad cuántica en optimización que no detallan el solver clásico de referencia ni el tamaño de las instancias (Vocal Media, n.d.). El teorema de *no free lunch* recuerda que ningún optimizador domina sobre todas las clases de problemas (Wolpert & Macready, 1997), de modo que la comparación solo tiene sentido instancia a instancia.
- **Foco en la viabilidad teórica más que en la aplicabilidad práctica**: los análisis de ventaja cuántica suelen formularse en términos asintóticos o bajo supuestos ideales de hardware, sin considerar las restricciones reales de los dispositivos NISQ (Preskill, 2018).
- **Representación implícita**: el feature map o la formulación QUBO se tratan a menudo como un detalle de implementación, aunque determinan qué estructura puede explotar el algoritmo (Huang et al., 2021; Lucas, 2014).

Este trabajo responde a estas limitaciones con un mismo protocolo de preparación y medida para modelos clásicos y cuánticos, con baselines clásicos de referencia y con la representación como eje explícito de análisis.

3. Objetivos

El estado de la técnica del capítulo anterior sitúa la brecha entre promesas teóricas del QML / optimización cuántico-inspirada y lo medible en régimen NISQ (Preskill, 2018; Bowles et al., 2024). Este capítulo fija el objetivo general y los específicos del estudio.

3.1 Objetivo general

El objetivo general de este trabajo es **evaluar de forma empírica y sistemática** si los algoritmos cuánticos disponibles en la era NISQ ofrecen una **ventaja real y medible** frente a los algoritmos clásicos consolidados en problemas representativos de la ciencia de datos, con el fin de determinar las condiciones bajo las cuales la computación cuántica puede aportar valor práctico en este dominio (Preskill, 2018; Biamonte et al., 2017).

3.2 Objetivos específicos

1. **Definir un marco conceptual** que organice el estudio en torno a **clasificación**, **optimización** y **representación** (feature map / QUBO–Ising) (Lloyd et al., 2020; PennyLane, n.d.c; Parzygnat et al., 2025).

2. **Diseñar e implementar pipelines de clasificación** comparables entre modelos clásicos y cuánticos bajo un **protocolo justo** de partición, escalado y reducción dimensional cuando el presupuesto de qubits lo exija (Schuld & Killoran, 2019; Havlíček et al., 2019; Bowles et al., 2024).

3. **Diseñar e implementar pipelines de optimización combinatoria** comparables entre solvers clásicos y cuántico-inspirados (incluidos QAOA y VQE) sobre **formulaciones QUBO/Ising compartidas** (Farhi et al., 2014; Peruzzo et al., 2014; Lucas, 2014; Lubinski et al., 2024).

4. **Evaluar calidad de la solución y esfuerzo computacional** con métricas comunes, de forma **reproducible** (Bowles et al., 2024; Cerezo et al., 2021).

5. **Interpretar los resultados** en el régimen NISQ de pocos qubits y simulación clásica, **delimitando qué se puede y no se puede afirmar** sobre ventaja cuántica (Huang et al., 2021; Thanasilp et al., 2024; McClean et al., 2018; Preskill, 2018).

4. Metodología y desarrollo experimental

Este capítulo describe cómo se ejecutaron los experimentos del TFM: entorno software, diseño experimental, datasets, preparación, representación en el espacio de Hilbert (o QUBO/Ising), modelos/solvers, y el protocolo de medición de calidad y latencia (Bowles et al., 2024).

4.1 Entorno tecnológico

La Tabla 4.1 recoge el software del estudio y sus versiones.

Capa	**Stack**	**Versión**
Lenguaje	Python	3.14
Notebooks	Jupyter (`ipykernel`, `nbformat`)	7.4; 5.11
Numérico / datos	NumPy, SciPy, pandas, h5py	2.5; 1.18; 3.1; 3.16
ML clásico	scikit-learn (SVM, Random Forest, PCA, métricas, splits); XGBoost (Chen & Guestrin, 2016)	1.9.1; 3.1.2
QML / circuitos	PennyLane (`default.qubit`, `lightning.qubit`) (PennyLane, n.d.m); Qiskit, Qiskit Aer (statevector) y `qiskit-machine-learning` (kernel de fidelidad) (Qiskit contributors, 2026)	0.45.1; 2.6.1; 0.18.2
QNN híbrida	PyTorch (`TorchLayer` + cabeza lineal)	2.15
Optimización	PuLP + CBC (ILP); NetworkX (grafos); annealing simulado propio sobre QUBO	2.9.0; 3.7
Visualización	matplotlib y utilidades propias de storytelling	3.11

Cuadro 4.1: Entorno software del estudio, con versiones en el orden en que aparecen los paquetes. Python es la versión del entorno virtual de ejecución; las demás versiones son las fijadas en las dependencias del repositorio, que no fijan `qiskit-machine-learning`.
Fuente: elaboración propia.

Todo el cómputo cuántico de este TFM es **simulación** de estado sobre CPU, sin hardware ruidoso (Preskill, 2018).

4.2 Diseño experimental

El estudio se organiza en tres ejes. Dos son familias de casos de uso; el tercero, la representación, es el paso que lleva el dato o el problema al espacio donde opera el método cuántico, y está presente en ambas (Lloyd et al., 2020; PennyLane, n.d.c; Parzygnat et al., 2025).

Eje	**Pregunta**	**Casos del estudio**
Clasificación (*qué*)	¿Qué decisión tomar sobre el dato?	Iris, Bars and Stripes, Ising
Optimización (*cómo*)	¿Cuál es la mejor configuración?	TSP, MDS, MaxCut
Representación	¿En qué espacio se resuelve?	Feature map; QUBO–Ising

Cuadro 4.2: Ejes del estudio. La representación no es una tercera familia de datos: es la transformación que comparten clasificación y optimización.
Fuente: elaboración propia.

En clasificación, la representación es el feature map $\mathbf{x} \mapsto |\phi(\mathbf{x})\rangle$ (Sección 4.3.3). En optimización, es la reformulación del problema combinatorio como QUBO o Hamiltoniano de Ising (Sección 4.4.2) (Lucas, 2014).

Sobre esos ejes se fijan dos familias con la misma lógica de comparación justa (Bowles et al., 2024; XanaduAI, n.d.):

1. **Clasificación:** mismos splits, mismo escalado y, cuando $d > 4$, misma PCA a 4 componentes para *todos* los modelos (clásicos y cuánticos).

2. **Optimización:** misma instancia por problema; mismo óptimo de referencia; solvers clásicos y cuántico-inspirados evaluados con las mismas métricas de calidad y tiempo.

La Figura 4.1 resume el recorrido común a las dos familias.

4.3 Clasificación

4.3.1 Datasets

Los tres datasets difieren en el origen del dato. Iris y Bars and Stripes son datasets *convencionales*: el dato es clásico y cualquier estructura cuántica la introduce el feature

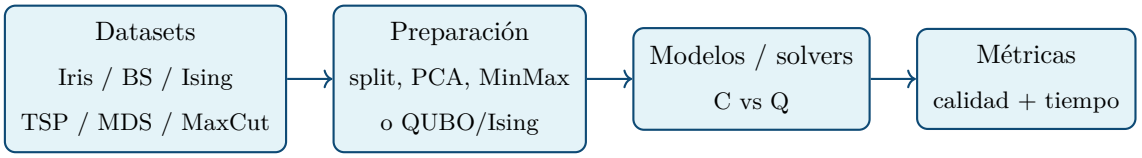


Figura 4.1: Pipeline experimental común: dato → preparación → modelo/solver → métricas agregadas.
Fuente: elaboración propia.

map. Ising es dato *de sistema*: cada muestra procede del estado fundamental de un Hamiltoniano y la etiqueta es su fase (PennyLane, n.d.n,n,n). La Tabla 4.3 resume tamaño, dimensión y partición de cada uno.

Dataset	**Muestras**	d	**Clases**	**Train / test y reducción**
Iris	150	4	3	90 / 60; sin PCA
Bars and Stripes	1200	16	2	720 / 480; PCA a 4 (57.5 % de varianza)
Ising 1D	200	32	2	120 / 80; PCA a 4 (99.9 % de varianza)

Cuadro 4.3: Datasets de clasificación. Partición estratificada con 40 % de test y semilla 123.
Fuente: elaboración propia.

Iris. Benchmark clásico de 150 muestras, 4 características continuas y 3 clases (scikit-learn developers, n.d.). Se carga con `sklearn.datasets.load_iris`. Con $d = 4$ cabe en 4 qubits **sin PCA**. La Figura 4.2 es el dato antes de cualquier clasificador: las cuatro variables proyectadas solo para verlas. Una clase queda separada; las otras dos se solapan.

Bars and Stripes (BS). Patrones binarios 4×4 del catálogo PennyLane (`bars-and-stripes`) (PennyLane, n.d.a,n). Es un dataset *convencional* etiquetado para QML, no un "quantum dataset" de sistema (PennyLane, n.d.n). Tras aplanar, $d = 16$; se comprime con PCA a 4, que retiene el 57.5 % de la varianza. La Figura 4.3 muestra el patrón antes del aplanado y la Figura 4.4, las dos clases ya proyectadas: comparten la misma región del plano.

Ising (fases). Datos PennyLane `qspin` / Ising (PennyLane, n.d.h): amplitudes del ground state (parte real e imaginaria) de cadenas 1D de cuatro espines (Figura 4.5). La

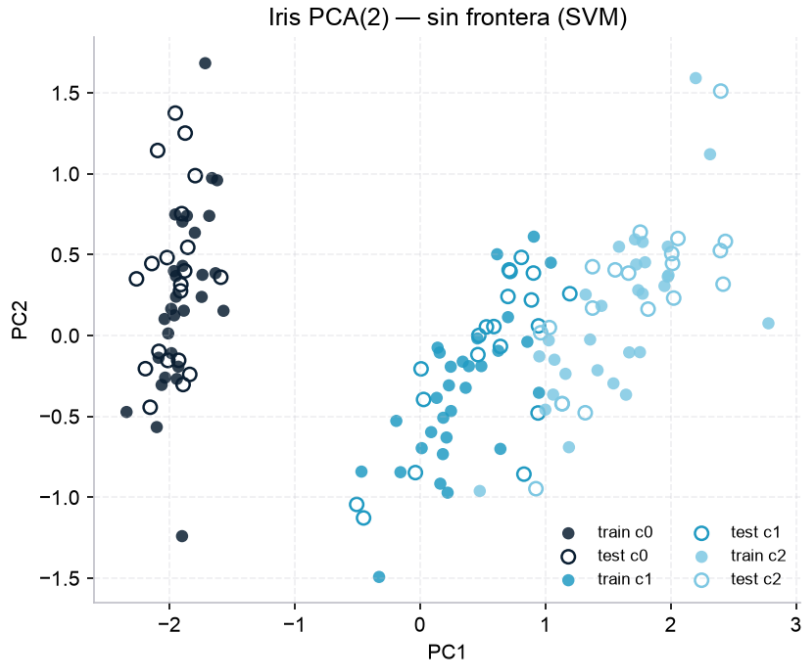


Figura 4.2: Iris antes de los clasificadores. Proyección visual de las cuatro variables originales; el modelo no usa esta PCA. Círculos huecos: test del protocolo.

Fuente: elaboración propia.

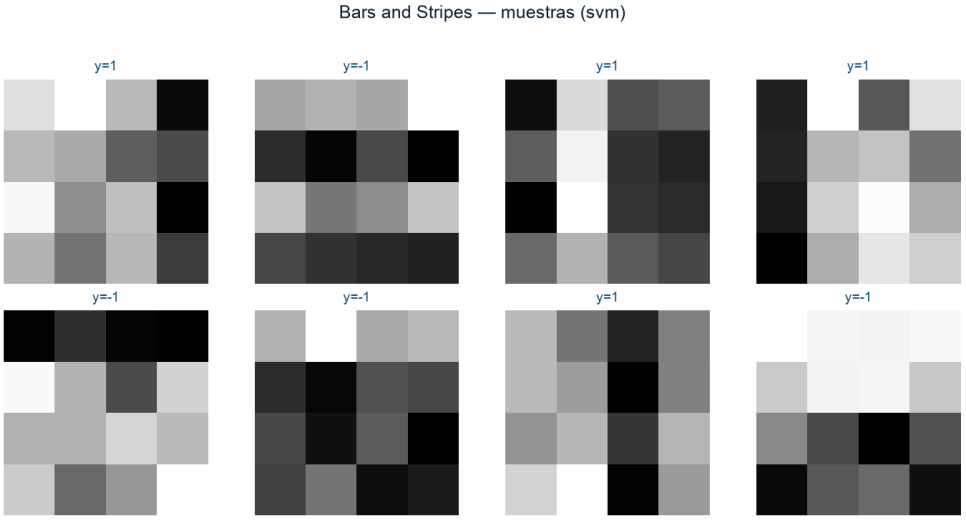


Figura 4.3: Bars and Stripes antes de aplanar y antes de cualquier clasificador. Cada panel es un patrón 4×4; $y \in \{+1, -1\}$ es la clase del catálogo.

Fuente: elaboración propia.

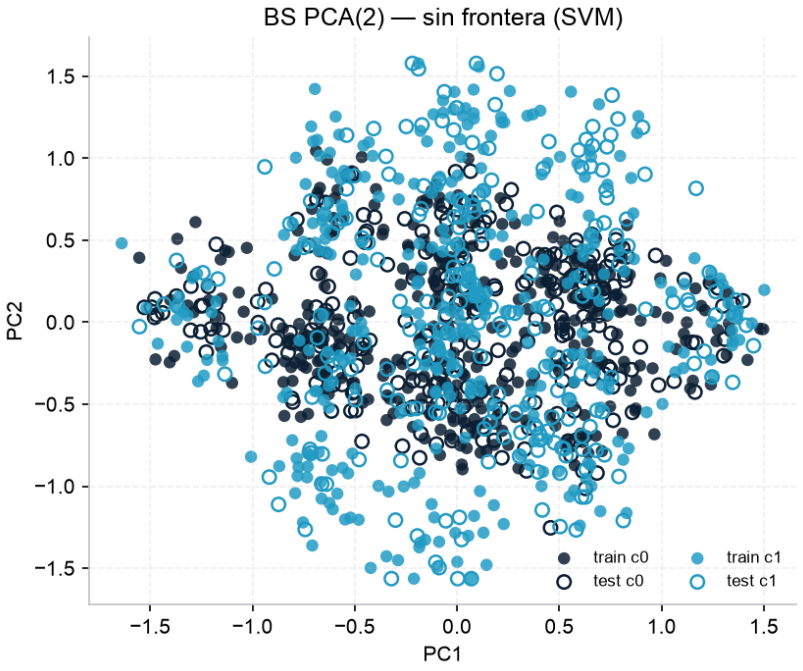


Figura 4.4: Bars and Stripes tras la reducción a componentes y antes de cualquier clasificador. Las dos clases comparten el mismo plano. Círculos huecos: test.

Fuente: elaboración propia.

etiqueta de fase se deriva del campo transversal, $y = \mathbf{1}[h > |J|]$; h **no** entra como feature, para evitar fuga de etiqueta. Las 2^4 amplitudes complejas dan $d = 32$ features reales, que la PCA reduce a 4.

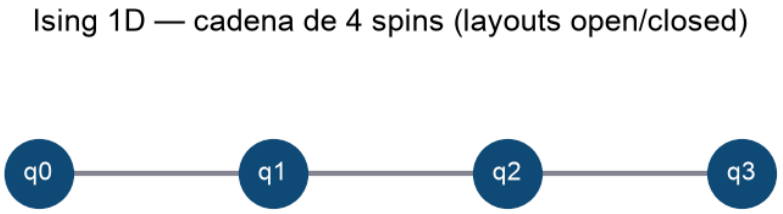


Figura 4.5: Sistema de Ising: cadena 1D de cuatro espines con acoplamiento J entre vecinos y campo transversal h, en layout abierto o cerrado. Las features son las amplitudes del estado fundamental de este Hamiltoniano para cada h.

Fuente: elaboración propia.

4.3.2 Preparación compartida

Protocolo fijo en los tres datasets:

1. `train_test_split` estratificado, `test_size`=0.4, `random_state`=123.
2. Si $d > 4$: `PCA(n_components=4, random_state=123)` ajustada solo en train y aplicada a test.
3. `MinMaxScaler(feature_range=(0, π))` ajustado en train (rango compatible con angle / ZZ encoding) (PennyLane, n.d.c).

La PCA, cuando existe, se aplica a clásicos y cuánticos por igual: se compara el modelo, no un preproceso distinto (Bowles et al., 2024).

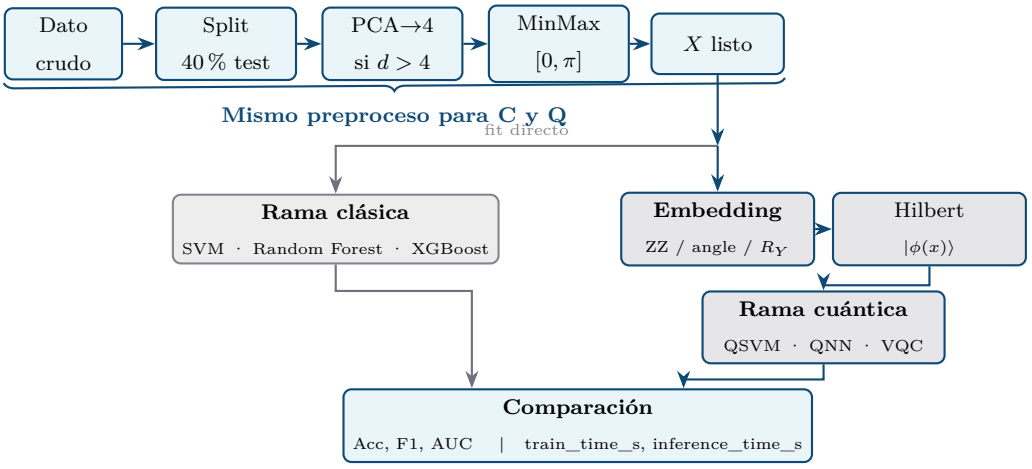


Figura 4.6: Preparación compartida y bifurcación C/Q. Tras split, PCA (si $d > 4$) y MinMax$[0, \pi]$, el mismo X alimenta baselines clásicos; en la rama cuántica pasa antes por el embedding al espacio de Hilbert. Ambas ramas se evalúan con las mismas métricas de calidad y latencia.

Fuente: elaboración propia.

4.3.3 Representación en el espacio de Hilbert

El feature map es un mapeo $\phi\colon \mathcal{X} \to \mathcal{H}$ que toma los datos en su espacio original $\mathcal{X} \subseteq \mathbb{R}^d$ y los transforma a estados en el espacio de Hilbert $\mathcal{H} = \left(\mathbb{C}^2\right)^{\otimes n} \simeq \mathbb{C}^{2^n}\colon \mathbf{x} \mapsto |\phi(\mathbf{x})\rangle$ (Lloyd et al., 2020; PennyLane, n.d.d; Schuld & Killoran, 2019; Parate, 2025). Tras PCA (si $d > n$) y, cuando el mapa es angular o ZZ, MinMax$[0, \pi]$, cada muestra se prepara con un circuito unitario $U_\phi(\mathbf{x})$ tal que $|\phi(\mathbf{x})\rangle = U_\phi(\mathbf{x})|0\rangle^{\otimes n}$. Sin ese embedding el dato

clásico no es un estado cuántico. Según Huang et al. (2021), la utilidad de ϕ depende de cómo se ajusta la geometría inducida en $\mathcal{H}$ a la estructura de las etiquetas (PennyLane, n.d.c; Parzygnat et al., 2025). Con presupuesto $n = 4$, $\mathcal{H} \simeq \mathbb{C}^{16}$. Se consideran cuatro embeddings:

- **Basis encoding.** Codifica cadenas binarias (o categorías discretizadas) en la base computacional: para $\mathbf{b} \in \{0,1\}^n$, $|\phi(\mathbf{b})\rangle = |b_1 \cdots b_n\rangle = \bigotimes_i X^{b_i}|0\rangle$ (Parate, 2025; PennyLane, n.d.d,n). Los estados son ortogonales; una medida en Z recupera $\mathbf{b}$ con certeza. Escala como $\lceil \log_2 C \rceil$ qubits para C categorías, pero no es natural para features continuas sin cuantización previa.

- **Mapa ZZ (`ZZFeatureMap`).** Preparación producto $H^{\otimes n}$ seguida de fases de ángulo $2x_i$ en cada qubit y puertas de fase entrelazadas $e^{-i\,\theta_{ij} Z_i Z_j}$ con $\theta_{ij} \propto (\pi - x_i)(\pi - x_j)$ (implementación `zz_feature_map` de Qiskit, `entanglement=full`) (Havlíček et al., 2019). El producto interno en $\mathcal{H}$ define el kernel de fidelidad $K(\mathbf{x}, \mathbf{x}') = |\langle\phi(\mathbf{x})|\phi(\mathbf{x}')\rangle|^2$.

- **Angle embedding.** `AngleEmbedding` aplica rotaciones locales $R_\sigma(x_i)$ ($\sigma \in \{X, Y, Z\}$) sobre el qubit i, de modo que $|\phi(\mathbf{x})\rangle = \bigotimes_{i=1}^{n} R_\sigma(x_i)|0\rangle$ (PennyLane, n.d.d,n; Parate, 2025). Codificación angular separable: no introduce entrelazamiento en la preparación del estado.

- **Preparación $R_Y(x_i)$.** Caso particular de angle embedding con $\sigma = Y$: $U_\phi(\mathbf{x}) = \bigotimes_i R_Y(x_i)$, un parámetro de rotación por dimensión de feature (PennyLane, n.d.m; Cerezo et al., 2021).

Las capas parametrizadas posteriores (`BasicEntanglerLayers`, `Rot`+CNOT), el estimador de K y la cabeza de clasificación no forman parte de ϕ: pertenecen al modelo. En los experimentos de clasificación, tras el protocolo compartido (PCA + MinMax$[0, \pi]$), se emplean mapa ZZ, angle embedding y R_Y; el basis encoding queda como alternativa para datos ya binarios o categóricos (p. ej. Bars and Stripes antes de la compresión) (Parate, 2025).

4.3.4 Modelos

Familia	Modelo	Configuración
Clásica	SVM (RBF) (Cortes & Vapnik, 1995)	`SVC` de scikit-learn, hiperparámetros por defecto
Clásica	Random Forest (Breiman, 2001)	100 árboles
Clásica	XGBoost (Chen & Guestrin, 2016)	100 árboles, profundidad 3, tasa de aprendizaje 0.1
Cuántica	QSVM (Havlíček et al., 2019)	mapa ZZ (2 repeticiones, entrelazamiento completo), kernel de fidelidad en statevector y `SVC`
Cuántica	QNN (PennyLane, n.d.l)	angle embedding, 6 `BasicEntanglerLayers`, cabeza lineal PyTorch; Adam ($\eta = 0.01$), 30 épocas, lotes de 32
Cuántica	VQC (PennyLane, n.d.m)	preparación $R_Y(x_i)$, 2 capas `Rot`+CNOT; Nesterov ($\eta = 0.1$), 100 iteraciones, lotes de 5

Cuadro 4.4: Modelos de clasificación. La configuración es la misma en los tres datasets. *Fuente: elaboración propia.*

Los seis clasificadores se evalúan sobre las mismas particiones y el mismo $\mathbf{x}$ post-preproceso. La rama clásica opera directamente en $\mathbb{R}^n$. En la rama cuántica, cada modelo fija un U_ϕ y una regla de decisión sobre $|\phi(\mathbf{x})\rangle$: el QSVM estima la matriz Gram de fidelidad $K(\mathbf{x}, \mathbf{x}') = |\langle\phi(\mathbf{x})|\phi(\mathbf{x}')\rangle|^2$ y la usa como kernel de un `SVC`; la QNN y el VQC componen U_ϕ con un ansatz variacional y clasifican a partir de valores esperados $\langle Z_i\rangle$ (con cabeza lineal entrenable en el caso híbrido QNN) (Havlíček et al., 2019; PennyLane, n.d.m; Cerezo et al., 2021). Así, ϕ define la representación; el modelo define cómo se lee esa representación.

4.3.5 Métricas de clasificación y latencia

Tras el ajuste, la calidad se mide en el conjunto de test; el conjunto de train entra solo en el gap de generalización. La Tabla 4.5 fija las definiciones. F1 macro es la métrica de ordenación entre modelos.

Métrica	Definición
Accuracy	Fracción de etiquetas correctas, $\mathrm{Acc} = \frac{1}{N}\sum_i \mathbf{1}[\hat{y}_i = y_i]$.
Precision, recall	Medias macro sobre las clases.
F1	Media armónica macro de precision y recall.
AUC	Área bajo la curva ROC cuando el modelo produce scores.
Gap de generalización	$\mathrm{Acc}_{\mathrm{train}} - \mathrm{Acc}_{\mathrm{test}}$.
Latencia	Wall-clock de entrenamiento (`fit`) y de inferencia sobre el test.

Cuadro 4.5: Métricas de calidad y latencia en clasificación.

Fuente: elaboración propia.

4.4 Optimización

4.4.1 Problemas e instancias

Las tres instancias son pequeñas a propósito: la fuerza bruta da el óptimo $C^\star$ de referencia y cabe en el presupuesto de qubits de QAOA y VQE. La Tabla 4.6 las resume.

Problema	Instancia	Origen	Representación
TSP	5 nodos, 8 aristas	instancia propia	QUBO sobre 6 rutas candidatas
MDS	6 vértices, 6 aristas	instancia propia	QUBO con penalización de cobertura
MaxCut	6 vértices, 12 aristas	HamLib (Sawaya et al., 2024)	Ising sin restricciones

Cuadro 4.6: Instancias de optimización.

Fuente: elaboración propia.

TSP restringido. TSP y MDS son NP-difíciles en su versión general (Garey & Johnson, 1979); aquí se usan instancias pequeñas con óptimo verificable. El grafo tiene 5 nodos y 8 aristas con distancias fijas (Figura 4.7); las dos aristas ausentes, 1–4 y 3–4, se penalizan con coste 100. La ruta sale y vuelve al nodo 0 y se fija además que el nodo 2 sea la segunda ciudad visitada. Quedan $3! = 6$ rutas candidatas, y el óptimo $C^\star = 16$ es el de ese problema restringido. El QUBO asigna una variable binaria a cada ruta candidata, con una penalización de selección única (6 variables, 6 qubits) (Lucas, 2014). El ILP impone la misma restricción en su formulación de asignación de posiciones.

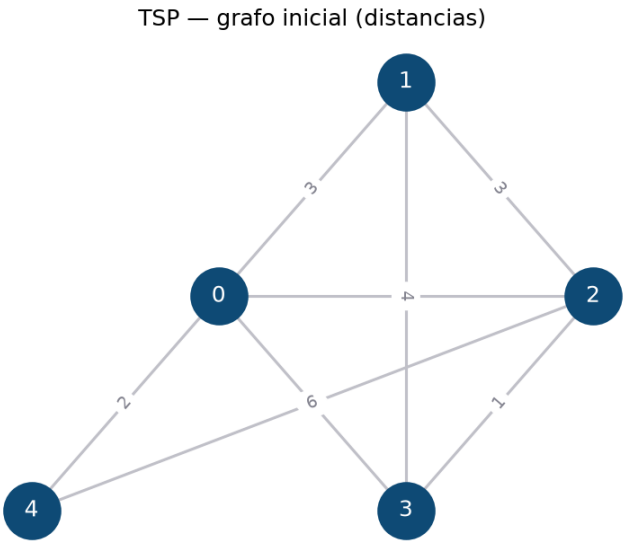


Figura 4.7: Instancia de TSP antes de cualquier solver. Cinco nodos, ocho aristas y sus distancias.

Fuente: elaboración propia.

Minimum Dominating Set (MDS). Grafo social de 6 vértices y 6 aristas (Figura 4.8); el objetivo es el conjunto dominante de tamaño mínimo. El ILP es el modelo de cobertura estándar. El QUBO es $A\sum_v x_v + B\sum_v \left(1 - \sum_{u \in N[v]} x_u\right)^2$, con $A = 1$, $B = 2$, una variable por vértice (6 qubits) y $N[v]$ la vecindad cerrada de v. La penalización actúa sobre cada vértice que queda sin dominar y también sobre cada vértice dominado más de una vez.

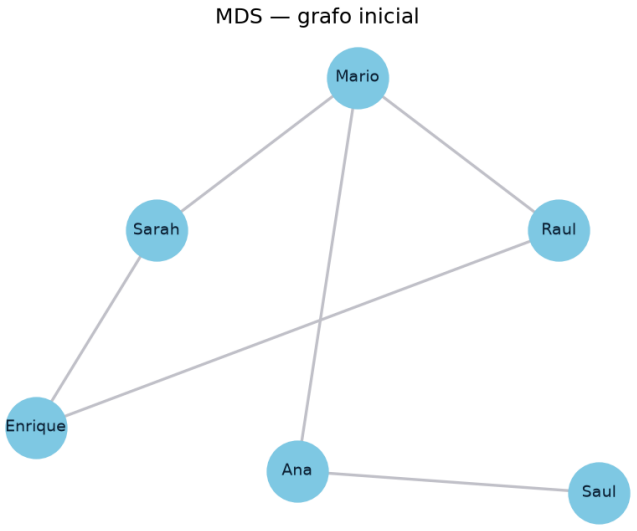


Figura 4.8: Instancia de MDS antes de cualquier solver. Seis vértices y seis aristas.

Fuente: elaboración propia.

MaxCut (HamLib). Instancia `circ_offsets-1-2` de HamLib: $n = 6$, 12 aristas y corte óptimo 8 (Figura 4.9) (Sawaya et al., 2024; PennyLane, n.d.f). El circuito QAOA sigue el demo de PennyLane para MaxCut (PennyLane, n.d.j; Farhi et al., 2014).

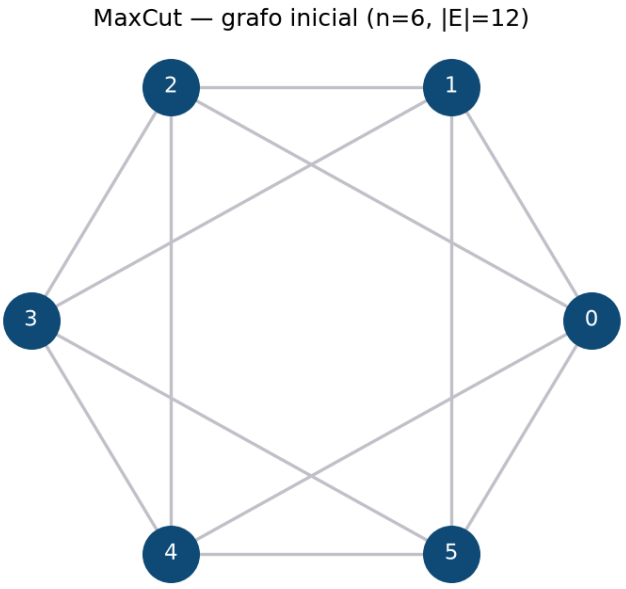


Figura 4.9: Instancia de MaxCut antes de cualquier solver. Seis vértices y doce aristas; el corte óptimo es 8.
Fuente: elaboración propia.

4.4.2 Preparación: de problema a QUBO/Ising

La reformulación QUBO–Ising es la cara de la representación en optimización: convierte el problema combinatorio en la minimización de una forma cuadrática sobre variables binarias, o de forma equivalente en la energía de un Hamiltoniano de espines H_C (Lucas, 2014). Las tres instancias no plantean la misma exigencia. MaxCut, tomado de HamLib, es el caso canónico de QAOA: el corte se escribe directamente como suma de términos Z_iZ_j y no tiene restricciones (Sawaya et al., 2024; PennyLane, n.d.f,n). TSP y MDS tienen restricciones (elegir una sola ruta candidata; dominar todos los vértices), que se introducen como penalizaciones. Ahí la codificación tiene que preservar a la vez el coste y la factibilidad.

La calidad de la representación importa tanto como el solver: un QUBO mal penalizado optimiza otra función (Lucas, 2014; Lubinski et al., 2024).

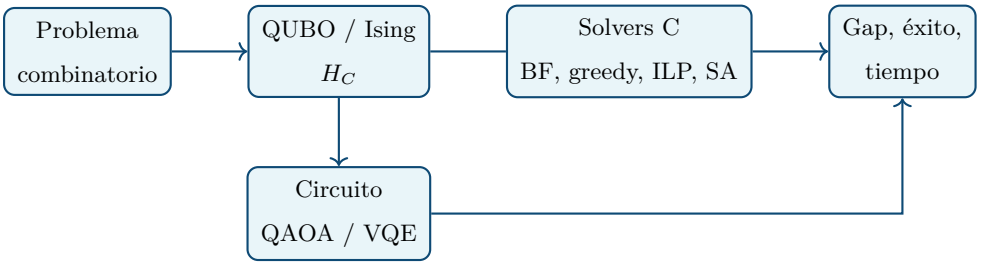


Figura 4.10: Optimización: la representación QUBO/Ising es compartida por SA, QAOA y VQE; BF/greedy/ILP operan sobre la formulación nativa o equivalente.
Fuente: elaboración propia.

4.4.3 Solvers

Solver	**TSP**	**MDS**	**MaxCut**
Fuerza bruta	6 rutas candidatas	2^6 subconjuntos	2^6 biparticiones
Greedy	Mejor prefijo $0 \to x \to 2$ y mejor cierre	Máxima cobertura de vecindad cerrada	Búsqueda local por volteo de un vértice
ILP (Gurobi Optimization, n.d.)	PuLP + CBC, asignación de posiciones	PuLP + CBC, cobertura	PuLP + CBC
QUBO + SA	20 semillas; 5000 iteraciones, $T_0 = 50$	20 semillas; 4000 iteraciones, $T_0 = 5$	20 semillas; 4000 iteraciones, $T_0 = 5$
QAOA (Farhi et al., 2014; PennyLane, n.d.i)	$p = 2$; 5 ejecuciones × 8 reinicios COBYLA; Qiskit Aer	$p = 3$; 8 semillas × 20 reinicios COBYLA; Qiskit Aer	$p = 2$; 8 ejecuciones, 40 pasos Adagrad; PennyLane
VQE (Peruzzo et al., 2014)	EfficientSU2; 5 ejecuciones × 6 reinicios COBYLA; Qiskit Aer	EfficientSU2; 5 ejecuciones × 6 reinicios COBYLA; Qiskit Aer	R_Y + anillo CNOT (2 capas); 5 ejecuciones × 3 reinicios COBYLA; PennyLane

Cuadro 4.7: Configuración de los solvers por problema. Fuerza bruta, greedy e ILP son deterministas y se ejecutan una vez; SA usa enfriamiento geométrico de razón 0.995.
Fuente: elaboración propia.

QAOA y VQE se simulan en statevector sobre CPU y leen la solución muestreando el estado final: en TSP y MDS toman el bitstring de menor energía QUBO entre 3000 o 4000 muestras; en MaxCut, VQE toma el de mayor corte entre 500 y QAOA, el más frecuente entre 200.

4.4.4 Métricas de optimización y latencia

La calidad se mide respecto del óptimo conocido $C^\star$ de cada instancia (TSP, MDS, MaxCut). La Tabla 4.8 fija las definiciones. Calidad y tiempo se leen juntos, como en los benchmarks de aplicación de Lubinski et al. (2024).

Métrica	**Definición**
Coste / corte	Valor medio y mejor de la función objetivo (longitud, tamaño del dominante o corte).
Gap absoluto	$\lvert C - C^\star \rvert$.
Gap relativo	$\lvert C - C^\star \rvert / \lvert C^\star \rvert$.
Success rate	Fracción de ejecuciones con $C = C^\star$.
Latencia	Tiempo medio wall-clock y, si el solver evalúa un Hamiltoniano, número medio de evaluaciones de energía.

Cuadro 4.8: Métricas de calidad y latencia en optimización. C es el coste o el corte obtenido; $C^\star$ es el óptimo de la instancia.
Fuente: elaboración propia.

4.5 Comparación clásico vs cuántico

Clásico y cuántico se comparan a lo largo de todo el ciclo de modelado: representación, elección del modelo o del solver, ajuste y lectura del resultado. Cada etapa tiene una lectura cualitativa, que fija la hipótesis del método, y una cuantitativa, que la mide con las métricas de las Tablas 4.5 y 4.8. La Tabla 4.9 detalla ambas lecturas por etapa (Bowles et al., 2024; Lubinski et al., 2024; Yu et al., 2026).

4.6 Reproducibilidad

El código, los datos de entrada, las métricas y las figuras están en el repositorio público

`https://github.com/juandapradam12/UNIR-MasterQC-QuantumAdvantageInDataScience`

Cada dataset o instancia tiene un notebook por modelo o solver; una carpeta de resultados por familia agrega las métricas y genera las figuras comparativas. El Apéndice A.1 describe la estructura del repositorio, el formato de las salidas y los pasos de reproducción.

Etapa	Cualitativo	Cuantitativo
Representación	En clasificación, $\mathbf{x} \in \mathbb{R}^n$ frente a $\|\phi(\mathbf{x})\rangle$; en optimización, formulación nativa frente a QUBO/Ising.	Dimensión de $\mathcal{H}$ ($n = 4$) y número de variables binarias.
Modelo / solver	Regla de decisión: kernel, árboles, boosting, ansatz variacional, enumeración, ILP o annealing.	Familia (clásica, cuántica o cuántico-inspirada) y criterio de parada.
Ajuste	Mismo split y escalado, o la misma instancia y el mismo $C^\star$.	Wall-clock de entrenamiento u optimización; evaluaciones de energía cuando el solver las cuenta.
Lectura	Generalización del clasificador, o distancia de la solución a $C^\star$, junto con su coste temporal.	Accuracy, F1, AUC, gap de generalización; coste, gap a $C^\star$, success rate y latencia.

Cuadro 4.9: Ciclo de modelado comparado. La columna cualitativa delimita qué es común y qué es propio de cada familia; la cuantitativa aplica las métricas de calidad y latencia. *Fuente: elaboración propia.*

Las semillas están fijadas en las particiones y en los solvers estocásticos. Cada notebook escribe sus métricas en JSON; los notebooks de resultados solo leen esos JSON, generan las tablas resumen y las figuras, y las cifras de este documento salen de ellos. Las versiones de las dependencias están fijadas en el propio repositorio, salvo `qiskit-machine-learning`.

4.7 Limitaciones del diseño

- El cómputo cuántico es simulación de estado ideal. No hay ejecución en hardware cuántico, así que ruido, conectividad del chip y tiempo de cola quedan fuera del diseño (Preskill, 2018).
- El presupuesto es de 4 qubits en clasificación y 6 en optimización. El alcance es ese régimen de pocas variables (Thanasilp et al., 2024).
- Los hiperparámetros quedan fijados por el protocolo (Tablas 4.4 y 4.7), sin barrido.
- Cada modelo se entrena sobre una única partición; no hay intervalos de confianza en clasificación. Con 60 muestras de test en Iris, un acierto equivale a 0.017 de accuracy.

- Los tiempos son wall-clock de una sola máquina. En los solvers deterministas proceden de una única ejecución y varían entre ejecuciones: el ILP del TSP pasó de 1.28 s a 0.07 s al repetir el notebook. Solo los órdenes de magnitud son comparables.

5. Resultados y análisis

Este capítulo aplica el protocolo del Capítulo 4 y responde a la pregunta del trabajo: en simulación de pocos qubits, qué calidad obtienen los métodos cuánticos y cuántico-inspirados frente a sus referentes clásicos, y a qué coste. Cada familia se presenta igual. Primero va una tabla con las seis implementaciones; después, una lectura por implementación que separa lo medido de lo que el experimento no permite atribuir; al final, una lectura conjunta (Huang et al., 2021; Cerezo et al., 2021; Lucas, 2014). Las cifras proceden de las métricas exportadas por los notebooks. Los tiempos se interpretan por orden de magnitud, por la variabilidad entre ejecuciones descrita en las limitaciones del diseño.

5.1 Clasificación

La Figura 5.1 resume la familia. En Iris y Bars and Stripes la media cuántica de accuracy queda 0.083 y 0.101 por debajo de la clásica; en Ising las dos medias coinciden (0.971). Ising es también el dataset en el que los seis modelos alcanzan al menos 0.95 de accuracy: la paridad aparece en el caso más fácil del corpus.

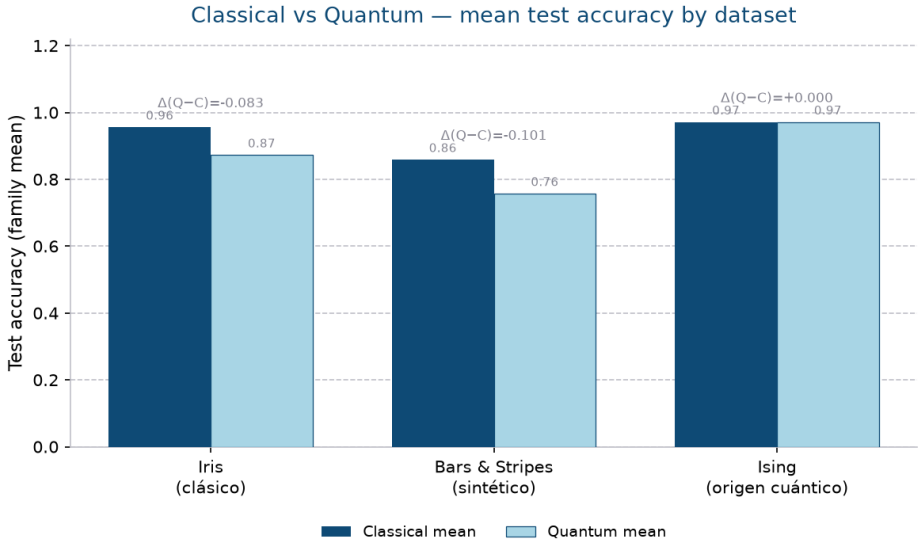


Figura 5.1: Accuracy de test media de la familia clásica (SVM, Random Forest, XGBoost) y de la cuántica (QSVM, QNN, VQC) en cada dataset. $\Delta(Q - C)$ es la diferencia entre ambas medias.

Fuente: elaboración propia.

5.1.1 Iris

Iris entra con $d = 4$, sin PCA. Una clase está separada y las otras dos se solapan (Figura 4.2). Con 60 muestras de test, cada error resta 0.017 de accuracy. La Tabla 5.1 recoge las métricas y la Figura 5.2, la frontera de cada modelo sobre las dos primeras componentes principales, que solo se usan para visualizar.

Modelo	**Acc**	**F1**	**AUC**	**Gap**	t_{train} (s)	t_{pred} (s)
SVM (RBF)	0.967	0.967	0.998	0.011	0.002	0.0004
Random Forest	0.950	0.950	0.991	0.050	0.065	0.005
XGBoost	0.950	0.950	0.983	0.050	0.155	0.002
QSVM	0.733	0.730	0.883	0.233	0.072	0.097
QNN	0.933	0.933	0.975	0.022	1.206	0.005
VQC	0.950	0.950	—	−0.006	49.305	0.151

Cuadro 5.1: Iris, conjunto de test. Gap = $\text{Acc}_{\text{train}} - \text{Acc}_{\text{test}}$. El VQC no produce scores y no tiene AUC.
Fuente: elaboración propia.

SVM. El kernel RBF traza tres regiones suaves y falla dos muestras de test, ambas en el solape entre versicolor y virginica. El gap es 0.011 y el ajuste dura 2 ms.

Random Forest y XGBoost. Empatan en F1 (0.950, un error más que el SVM) y ajustan el train por completo, de ahí el gap de 0.050. En el espacio de entrada sus fronteras son umbrales por variable; en la proyección de la Figura 5.2 aparecen como cortes casi rectos. Con 90 muestras de entrenamiento no mejoran al RBF (Breiman, 2001; Chen & Guestrin, 2016).

QSVM. Es el peor modelo del dataset: 16 errores de 60, accuracy de train 0.967 y un gap de 0.233, el mayor del corpus. En la Figura 5.2 el kernel de fidelidad fragmenta el plano en islas y falla también muestras de setosa, la clase que los otros cinco modelos clasifican sin error. El resultado es compatible con lo que describen Huang et al. (2021): un kernel cuántico cuya geometría no se ajusta al dato puede generalizar peor que uno clásico. El experimento muestra el sobreajuste, pero no aísla su causa.

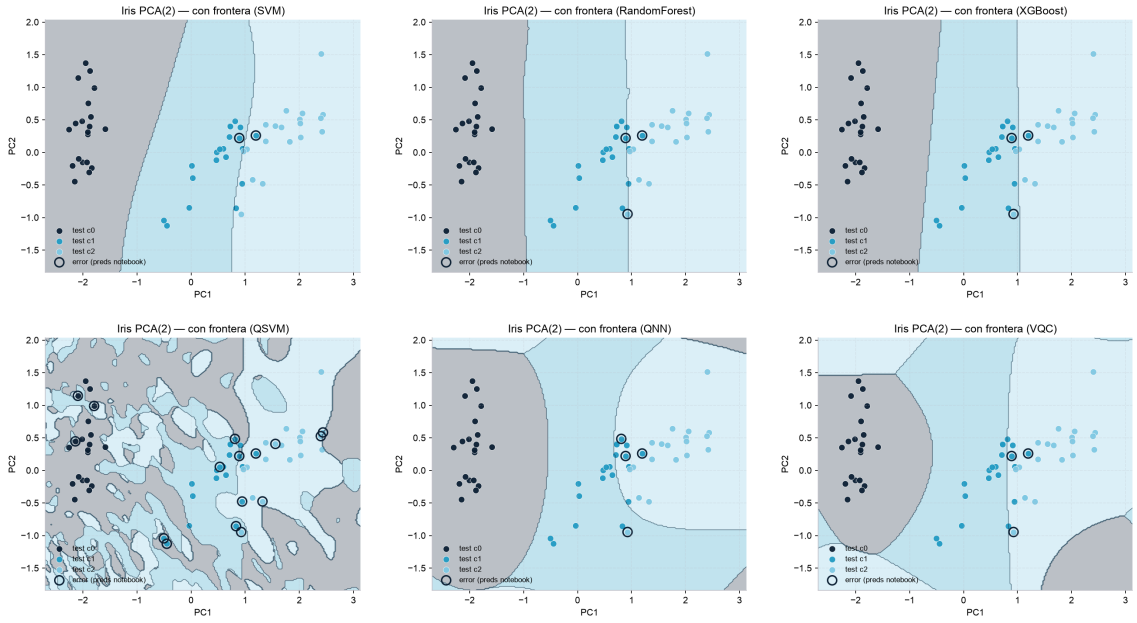


Figura 5.2: Fronteras de decisión en Iris. Cada panel evalúa el modelo entrenado sobre el plano de las dos primeras componentes principales de su entrada, con las demás direcciones fijas en la media del train. Fila superior: SVM, Random Forest y XGBoost; fila inferior: QSVM, QNN y VQC. Puntos: test; círculos: errores de test.

Fuente: elaboración propia.

QNN y VQC. Se acercan al clásico: la QNN queda a un error del bosque y el VQC lo iguala. Sus fronteras son suaves, sin la fragmentación del QSVM. La diferencia está en el coste: el VQC necesita 49 s de entrenamiento, cuatro órdenes de magnitud más que el SVM, para la calidad de los árboles (Cerezo et al., 2021; Bowles et al., 2024).

5.1.2 Bars and Stripes

El patrón 4×4 se aplana ($d = 16$) y la PCA a cuatro componentes retiene el 57.5 % de la varianza. En el plano principal las dos clases ocupan la misma región (Figura 4.4). Es el dataset con menor accuracy para cinco de los seis modelos (el QSVM rinde aún peor en Iris) y en el que la familia cuántica queda más lejos (Tabla 5.2). La Figura 5.3 muestra que las dos primeras componentes no bastan para separar las clases: el SVM asigna casi todo el plano a una sola clase y aun así acierta el 87 % del test, así que su decisión depende de la tercera y la cuarta componentes. Sobre ese mismo plano, el QSVM vuelve a fragmentar el espacio en islas.

Modelo	**Acc**	**F1**	**AUC**	**Gap**	t_{train} (s)	t_{pred} (s)
SVM (RBF)	0.871	0.870	0.936	0.021	0.004	0.004
Random Forest	0.854	0.854	0.943	0.146	0.079	0.007
XGBoost	0.850	0.849	0.953	0.067	0.064	0.001
QSVM	0.758	0.758	0.822	0.110	1.428	1.372
QNN	0.817	0.815	0.936	−0.006	4.825	0.005
VQC	0.696	0.688	—	0.014	12.432	1.872

Cuadro 5.2: Bars and Stripes, conjunto de test (480 muestras).

Fuente: elaboración propia.

SVM. Vuelve a dar el mejor F1 con un gap de 0.021. Opera sobre las mismas cuatro componentes que los circuitos, así que su ventaja no procede de ver más información.

Random Forest y XGBoost. Quedan 1.6 y 2.1 puntos por debajo del SVM en F1 y lo superan en AUC (0.943 y 0.953 frente a 0.936): ordenan mejor los scores aunque acierten menos etiquetas con el umbral por defecto. El bosque ajusta el train por completo (accuracy 1.0) y tiene el gap más alto del dataset (0.146). XGBoost, con árboles de profundidad 3, llega a 0.917 en train y su gap (0.067) es menos de la mitad.

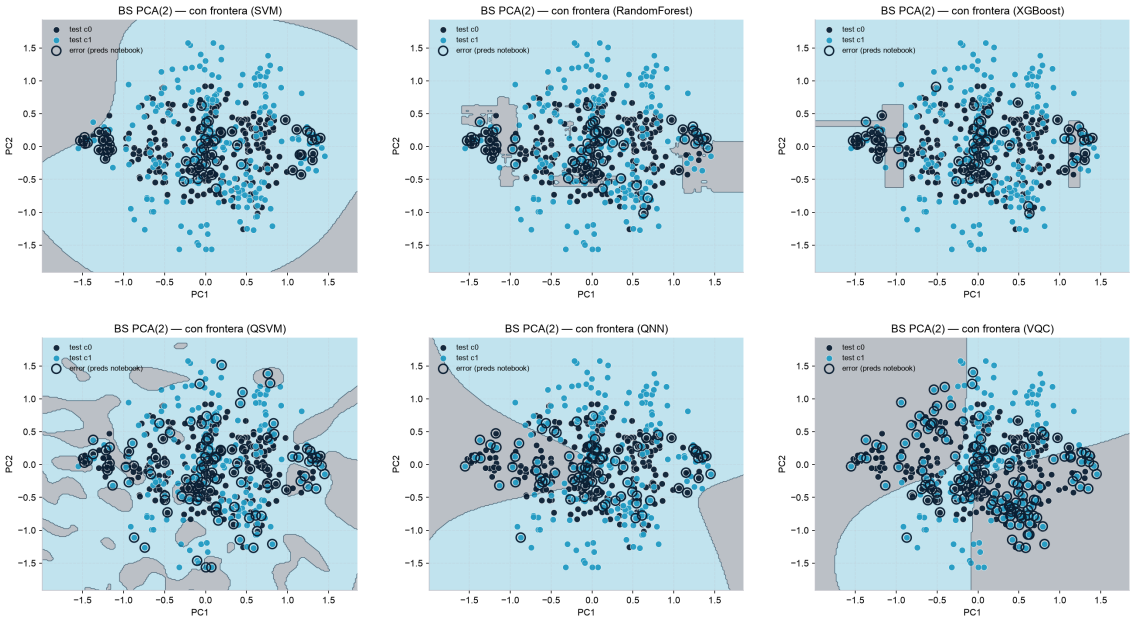


Figura 5.3: Fronteras de decisión en Bars and Stripes, con la misma construcción que la Figura 5.2. Las dos primeras componentes no separan las clases, así que el plano solo muestra una sección de cada frontera. Fila superior: SVM, Random Forest y XGBoost; fila inferior: QSVM, QNN y VQC.

Fuente: elaboración propia.

QSVM. F1 0.758, 11 puntos bajo el SVM, con un gap de 0.110. El coste cambia de escala: el ajuste evalúa el kernel sobre los 720×720 pares de train y la inferencia sobre los 480×720 pares test–train, y ambos pasan de milisegundos a 1.4 s (Schuld & Killoran, 2019).

QNN. Es el mejor modelo cuántico del dataset. Su AUC (0.936) es prácticamente la del SVM, aunque su F1 quede 5.5 puntos por debajo. Su accuracy de train (0.811) es menor que la de test (0.817): no hay sobreajuste, y el modelo tampoco ajusta mejor los datos con los que se entrena.

VQC. F1 0.688, el mínimo del corpus, con accuracy de train 0.710: el modelo no ajusta ni el train. Lee solo dos valores esperados, $\langle Z_0 \rangle$ y $\langle Z_1 \rangle$, tras dos capas `Rot`+CNOT. En Iris, con el mismo ansatz y tres valores esperados, igualaba a los árboles (Cerezo et al., 2021).

5.1.3 Ising

Las features son las amplitudes del estado fundamental de la cadena de la Figura 4.5, en sus versiones abierta y cerrada con 100 valores de h cada una, reducidas a cuatro componentes que retienen el 99.9 % de la varianza. La etiqueta es la fase, $y = \mathbf{1}[h > |J|]$. En el plano principal las muestras de cada cadena forman una curva y la fase cambia a lo largo de ella. Es el único dataset en el que un modelo cuántico empata con el mejor clásico (Tabla 5.3). Con 80 muestras de test, cada error resta 0.0125 de accuracy: las diferencias entre modelos son de uno a tres errores.

Modelo	**Acc**	**F1**	**AUC**	**Gap**	t_{train} (s)	t_{pred} (s)
SVM (RBF)	0.988	0.985	1.000	0.013	0.002	0.0004
Random Forest	0.975	0.969	1.000	0.025	0.044	0.005
XGBoost	0.950	0.937	0.996	0.050	0.062	0.001
QSVM	0.988	0.985	1.000	0.013	0.105	0.135
QNN	0.950	0.937	0.996	0.017	1.754	0.005
VQC	0.975	0.969	—	0.000	11.396	0.210

Cuadro 5.3: Ising, conjunto de test (80 muestras). El SVM y el QSVM fallan la misma cantidad de muestras (una).

Fuente: elaboración propia.

SVM, Random Forest y XGBoost. Los tres alcanzan AUC ≥ 0.996: las amplitudes cambian lo bastante con h como para que un margen o unos umbrales separen las fases. El SVM falla una muestra y XGBoost cuatro, todas de la fase ordenada ($h \leq |J|$), de ahí su recall macro de 0.917.

QSVM. Empata con el SVM en todas las métricas. El empate no indica que el mapa ZZ capture la física de la cadena: el mapa actúa sobre cuatro componentes principales de las amplitudes, no sobre los espines, y en el plano principal sigue fragmentando el espacio en islas, como en Iris. La paridad se da en el dataset más fácil del corpus. Tampoco se traslada al tiempo: el QSVM tarda 0.11 s en ajustar y 0.13 s en predecir, frente a 2 ms y 0.4 ms del SVM.

QNN y VQC. El VQC iguala al bosque (dos errores) y la QNN a XGBoost (cuatro). El VQC vuelve a ser el modelo más caro (11.4 s).

5.1.4 Lectura de la familia

La Figura 5.4 muestra la accuracy por modelo y dataset. Entre su peor y su mejor dataset, los clásicos varían de 10 a 12 puntos de accuracy y la QNN 13; el QSVM varía 25 y el VQC 28. QSVM y VQC son los modelos más sensibles al dataset: el QSVM es el peor en Iris y empata con el mejor en Ising.

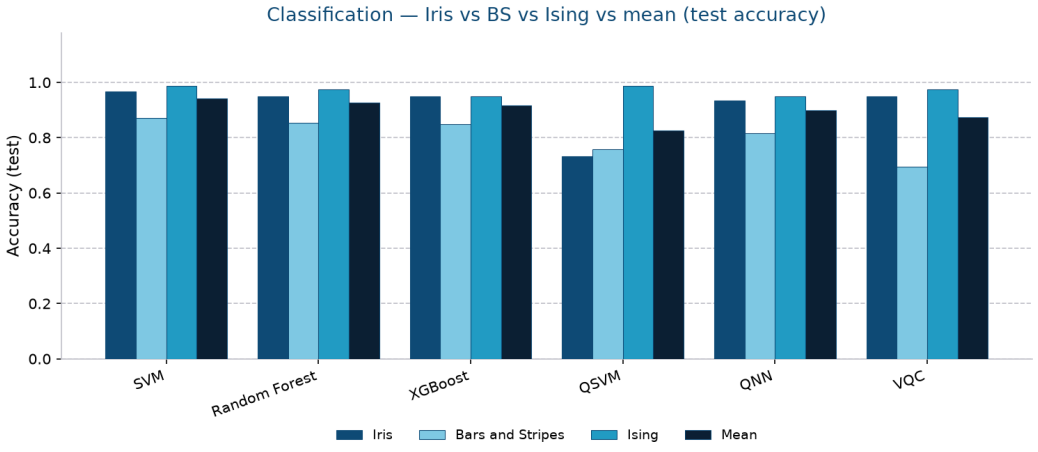


Figura 5.4: Accuracy de test por modelo en cada dataset y su media.
Fuente: elaboración propia.

La Figura 5.5 cruza calidad media y coste de entrenamiento. El SVM ocupa la esquina de mayor F1 medio (0.940) y menor tiempo (2 ms). Ningún modelo cuántico está en la

frontera de Pareto: la QNN, el mejor cuántico en media (F1 0.895), entrena tres órdenes de magnitud más despacio, y el VQC (F1 0.869) cuatro. En inferencia, QSVM y VQC son de dos a tres órdenes más lentos que los clásicos: el QSVM calcula la fidelidad de cada muestra de test con todo el train y el VQC simula el circuito muestra a muestra. La QNN predice en milisegundos.

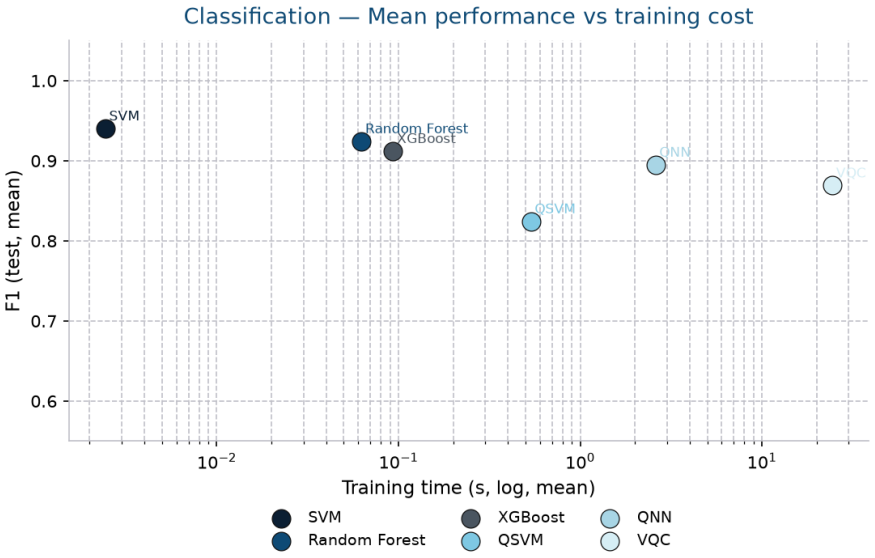


Figura 5.5: F1 de test medio sobre los tres datasets frente al tiempo medio de entrenamiento (escala logarítmica).

Fuente: elaboración propia.

5.2 Optimización

Las tres instancias tienen óptimo conocido y caben en enumeración. La Tabla 5.4 reúne los resultados y la Tabla 5.5 describe cada instancia como el espacio de 2^6 bitstrings que muestrean QAOA y VQE.

La forma de leer la solución condiciona cómo se interpreta el éxito de los solvers variacionales. En TSP y MDS, QAOA y VQE devuelven el bitstring de menor energía QUBO entre 3000 o 4000 muestras, y VQE en MaxCut el de mayor corte entre 500. Un muestreo uniforme de ese tamaño contendría el óptimo con probabilidad prácticamente 1 en las tres instancias, así que en esos casos un éxito de 1 no prueba que el circuito concentre probabilidad en el óptimo. Esa concentración solo se conoce en QAOA: en MDS el notebook registra la probabilidad de medir el óptimo, y en MaxCut la lectura es el bitstring más frecuente de 200 muestras.

	TSP ($C^\star = 16$)			**MDS** ($C^\star = 2$)			**MaxCut** ($C^\star = 8$)		
Solver	Éxito	$\bar{C}$	t (s)	Éxito	$\bar{C}$	t (s)	Éxito	$\bar{C}$	t (s)
Fuerza bruta	1.00	16.0	$6 \cdot 10^{-5}$	1.00	2.00	$2 \cdot 10^{-4}$	1.00	8.0	$3 \cdot 10^{-4}$
Greedy	1.00	16.0	$9 \cdot 10^{-5}$	0.00	3.00	$4 \cdot 10^{-5}$	1.00	8.0	$6 \cdot 10^{-5}$
ILP	1.00	16.0	0.070	1.00	2.00	0.050	1.00	8.0	0.657
QUBO + SA	0.90	16.3	0.017	1.00	2.00	0.007	1.00	8.0	0.003
QAOA	1.00	16.0	1.122	0.875	2.25	12.597	1.00	8.0	0.644
VQE	1.00	16.0	2.021	1.00	2.00	1.594	1.00	8.0	0.980

Cuadro 5.4: Resultados de optimización. $\bar{C}$ es el coste medio sobre ejecuciones (corte medio en MaxCut) y t, el tiempo medio por ejecución. Evaluaciones de energía por ejecución (TSP, MDS y MaxCut): SA 5001, 4001 y 4000; QAOA 356, 4000 y 40 pasos de Adagrad; VQE 720, 600 y 240. En VQE y en QAOA sobre MDS coinciden con el máximo permitido, reinicios × iteraciones de COBYLA.

Fuente: elaboración propia.

	TSP	**MDS**	**MaxCut**
Qubits / variables binarias	6	6	6
Restricciones penalizadas	selección única	cobertura	ninguna
Bitstrings factibles (de 64)	2	35	64
Bitstrings óptimos	1	1	18
P(óptimo) con una muestra uniforme	0.016	0.016	0.28

Cuadro 5.5: Cada instancia como espacio de 2^6 bitstrings. En el TSP solo son factibles los bitstrings que seleccionan una única ruta y que esa ruta no use aristas ausentes, lo que deja 2 de las 6 rutas candidatas. En MaxCut los 18 bitstrings óptimos forman 9 cortes distintos salvo complemento.

Fuente: elaboración propia.

5.2.1 TSP

Solo dos de las seis rutas candidatas son factibles, con costes 16 y 19. La ruta óptima, $[0, 4, 2, 3, 1, 0]$ (Figura 5.6), es la que devuelven los seis solvers; el SA, en 18 de sus 20 semillas.

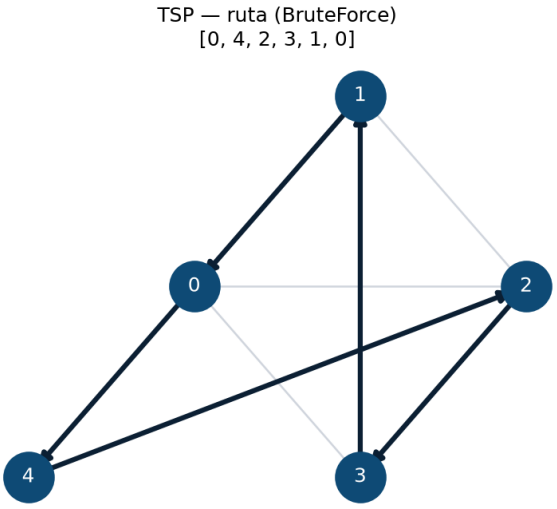


Figura 5.6: Ruta óptima del TSP restringido, de coste 16.
Fuente: elaboración propia.

Fuerza bruta, greedy e ILP. La enumeración evalúa seis rutas en 60 μs. El greedy elige el prefijo $0 \to x \to 2$ más barato y completa la ruta; en esta instancia eso basta. El ILP certifica el óptimo con CBC en 0.07 s, tiempo que incluye construir el modelo y llamar al solver externo. En este tamaño la enumeración es el método de referencia, no un baseline débil.

QUBO + SA. Acierta en 18 de 20 semillas. Las semillas 1 y 6 terminan en la otra ruta factible, $[0, 4, 2, 1, 3, 0]$, de coste 19. Con una variable por ruta, cualquier volteo de un bit desde una ruta factible rompe la selección única y suma al menos la penalización de 100, así que ambas rutas son mínimos locales para el SA. Cada ejecución hace 5001 evaluaciones en 17 ms. El SA opera sobre la misma energía que QAOA y VQE, y la enumeración del QUBO confirma que su mínimo es la ruta óptima (Lucas, 2014).

QAOA y VQE. Ambos devuelven la ruta óptima en las cinco ejecuciones. QAOA ($p = 2$) usa de media 356 evaluaciones, por debajo de su máximo de 640 (8 reinicios $\times$ 80

iteraciones), y 1.1 s; VQE agota sus 720 (6×120) en 2.0 s (Farhi et al., 2014; Peruzzo et al., 2014). Con la lectura por mínima energía entre 3000 muestras, el éxito no mide cuánta probabilidad concentran en el óptimo. Los dos reproducen un óptimo que la enumeración obtiene cuatro órdenes de magnitud antes.

5.2.2 Minimum dominating set

El óptimo, $\{\text{Ana}, \text{Enrique}\}$, es único: 35 de los 64 subconjuntos son dominantes y solo ese tiene tamaño 2. Es la única instancia en la que fallan el greedy y QAOA. La Figura 5.7 compara el óptimo con la solución del greedy.

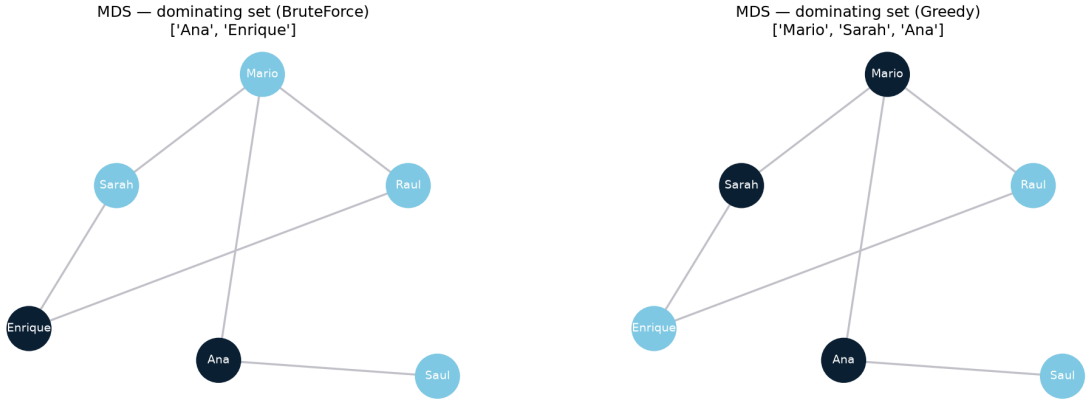


Figura 5.7: MDS. Izquierda: conjunto dominante mínimo, $\{\text{Ana}, \text{Enrique}\}$, devuelto por fuerza bruta, ILP, SA, VQE y siete de las ocho semillas de QAOA. Derecha: solución del greedy, de tamaño 3.
Fuente: elaboración propia.

Fuerza bruta e ILP. Ambos devuelven el óptimo; la enumeración en 0.2 ms y el ILP de cobertura en 0.05 s.

Greedy. Elige primero a Mario, que cubre cuatro vértices, y después necesita a Sarah y a Ana para cubrir a Enrique y a Saul: tamaño 3, éxito 0. La regla de máxima cobertura no deshace la primera elección, y el óptimo no contiene a Mario. Es el único caso en que fuerza bruta, greedy o ILP no alcanzan el óptimo, y no depende de la semilla: la regla es determinista.

QUBO + SA. Acierta en las 20 semillas en 7 ms. Sobre la misma formulación, con $A = 1$ y $B = 2$, el recocido clásico encuentra el óptimo; el QUBO no es la causa de los

fallos de QAOA (Lucas, 2014).

QAOA. Acierta en siete de las ocho semillas. La semilla fallida devuelve {Enrique, Saul}, que deja a Mario sin dominar: una solución infactible con energía QUBO $2 + B = 4$, de ahí el coste medio 2.25. Esa semilla termina con la energía esperada más alta de las ocho (-9.96) y con probabilidad nula de medir el óptimo. La siguiente peor (-10.04) acierta, pero mide el óptimo solo con probabilidad 0.11; las otras seis terminan entre -11.8 y -12.5 y lo miden con probabilidad entre 0.62 y 0.75. La probabilidad de medir el óptimo sigue, por tanto, a la energía alcanzada por el optimizador (media 0.53 en las ocho semillas). Con $p = 3$ y 20 reinicios de COBYLA, que agotan las 4000 evaluaciones, la semilla fallida termina en parámetros de mayor energía: el fallo está en la optimización de los parámetros, no en la formulación (Farhi et al., 2014; Lubinski et al., 2024). Es además el solver más lento de la familia (12.6 s).

VQE. Acierta en las cinco ejecuciones; agota sus 600 evaluaciones (6 reinicios $\times$ 100) en 1.6 s. Con la lectura por mínima energía entre 4000 muestras, el éxito no indica cuánta probabilidad concentra el estado en el óptimo, y el notebook no la registra. Es dos órdenes de magnitud más lento que el SA (Peruzzo et al., 2014).

5.2.3 MaxCut

Instancia HamLib de 6 vértices y 12 aristas, con corte óptimo 8. Los seis solvers lo alcanzan en todas las ejecuciones. El resultado es poco informativo por sí solo: 18 de las 64 asignaciones son óptimas (Tabla 5.5), así que una muestra uniforme acierta con probabilidad 0.28. Los solvers estocásticos devuelven biparticiones distintas con el mismo corte; la Figura 5.8 muestra la de la enumeración.

Clásicos. La enumeración recorre las 2^6 asignaciones en 0.3 ms. La búsqueda local del greedy, que voltea el vértice que más aumenta el corte, llega a un óptimo con un solo volteo desde una asignación aleatoria. El SA lo hace en las 20 semillas en 3 ms. El ILP tarda 0.66 s, frente a 0.07 s en el TSP; en una ejecución anterior el ILP del TSP tardó 1.28 s, así que estos tiempos no ordenan las instancias por dificultad.

QAOA y VQE. QAOA ($p = 2$) acierta en las ocho ejecuciones con 40 pasos de Adagrad y 0.64 s. Su lectura es el bitstring más frecuente de 200 muestras, y con los parámetros

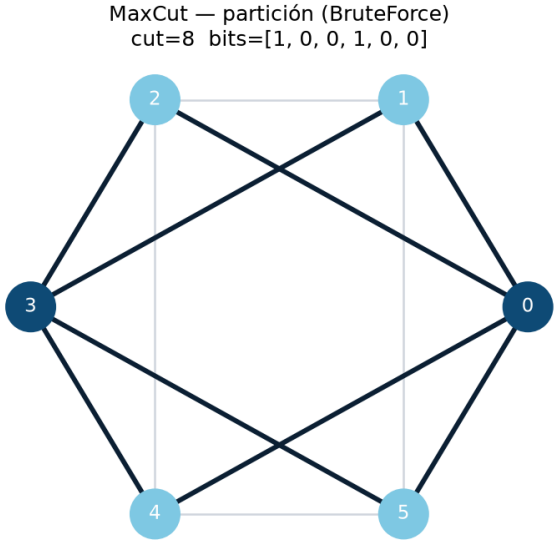


Figura 5.8: Un corte óptimo de MaxCut (valor 8). El óptimo es degenerado: 18 asignaciones, 9 cortes distintos salvo complemento.
Fuente: elaboración propia.

finales guardados el estado asigna probabilidad 0.93 a los cortes óptimos en las ocho ejecuciones, frente a 0.28 de una muestra uniforme: aquí sí hay concentración medida. MaxCut es el caso de uso estándar de QAOA: el Hamiltoniano de coste es el propio corte y no hay penalizaciones (Farhi et al., 2014; PennyLane, n.d.j). VQE acierta con 240 evaluaciones (3 reinicios × 80) y 0.98 s, con lectura del mejor corte entre 500 muestras.

5.2.4 Lectura de la familia

La Figura 5.9 resume calidad y tiempo medios. Los métodos exactos alcanzan $C^\star$ en las tres instancias; la enumeración, en menos de un milisegundo. El greedy solo falla en el MDS, por una regla que no rectifica. El SA sobre QUBO acierta siempre salvo en dos semillas del TSP, en milisegundos. QAOA y VQE recuperan $C^\star$ en todas las ejecuciones salvo una semilla de QAOA en el MDS, con tiempos de 0.6 a 2 s (QAOA en MDS, 12.6 s).

El único fallo cuántico aparece en el MDS. No se explica por la rareza del óptimo: en el TSP el óptimo también es un único bitstring de 64 y QAOA acierta en las cinco ejecuciones. Lo que se mide en el MDS es una semilla que termina con mayor energía esperada y probabilidad nula de medir el óptimo. Como la lectura en TSP y MDS toma el mejor de miles de bitstrings, el success rate de esos casos es una cota optimista; la probabilidad de medir el óptimo, registrada en QAOA sobre MDS y calculable en QAOA

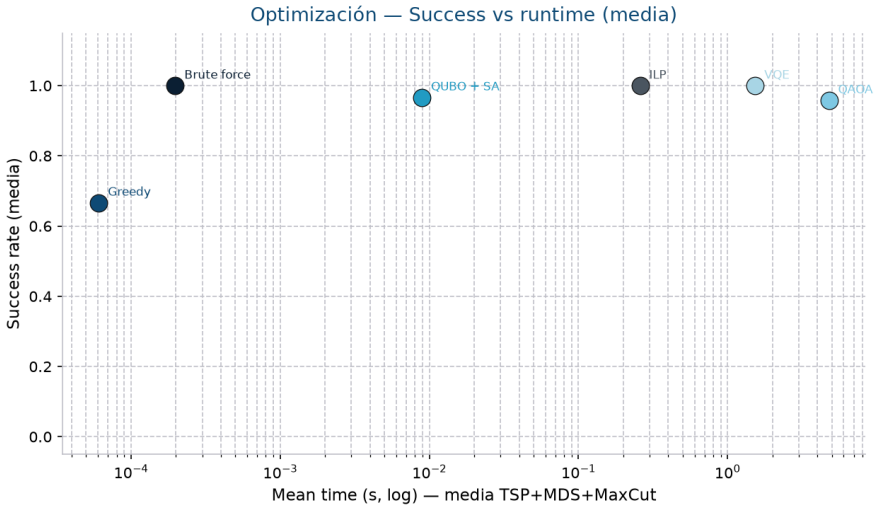


Figura 5.9: Success rate medio frente al tiempo medio por solver, promediado sobre TSP, MDS y MaxCut.

Fuente: elaboración propia.

sobre MaxCut a partir de los parámetros guardados, es la medida que distingue a los solvers. Con $n \leq 6$ el circuito no explora un espacio que la enumeración no recorra antes (Farhi et al., 2014; Lubinski et al., 2024).

5.3 Discusión: la representación como eje

Las dos familias se leen con la misma pregunta: en qué espacio se resuelve el problema y qué permite afirmar el experimento sobre ese espacio (Lloyd et al., 2020; Huang et al., 2021; Parzygnat et al., 2025).

En clasificación el mismo QSVM pasa de ser el peor modelo en Iris a empatar con el mejor en Ising. El experimento no permite atribuir ese cambio a una afinidad entre el mapa ZZ y el Hamiltoniano de Ising: el mapa actúa sobre componentes principales de las amplitudes, no sobre los espines, y en Ising los seis modelos alcanzan al menos 0.95 de accuracy. Lo que sí muestra es que, con el mismo preproceso, el mapa ZZ generalizó peor que un RBF en los dos datasets convencionales, en línea con Huang et al. (2021), y que ningún modelo cuántico superó al SVM en ningún dataset. Entre los cuánticos, la QNN es el más regular (13 puntos de rango de accuracy) y el VQC el menos (28) (Bowles et al., 2024).

En optimización la representación es común a SA, QAOA y VQE, y la enumeración del QUBO confirma que su mínimo es el óptimo de cada instancia; el SA lo alcanza en MDS y MaxCut y en 18 de 20 semillas del TSP. Las diferencias aparecen después, en si el optimizador del bucle variacional encuentra parámetros que concentren probabilidad en ese mínimo (Lucas, 2014; Lubinski et al., 2024).

Límite del régimen. Con 4 qubits en clasificación y 6 en optimización, el experimento mide cercanía al clásico, no ventaja asintótica (Preskill, 2018; Thanasilp et al., 2024). Los resultados dan paridad o déficit de calidad y un coste temporal mayor, que en simulación crece con 2^n. Quedan tres preguntas abiertas: si la paridad observada en Ising se mantiene al crecer la cadena y al codificar los espines directamente en qubits; cuánta probabilidad concentran QAOA y VQE en el óptimo cuando la lectura no puede apoyarse en miles de muestras de un espacio pequeño; y qué parte del coste se debe a la simulación y cuál al método.

6. Conclusiones

El trabajo partió de una pregunta: si los algoritmos cuánticos de la era NISQ ofrecen una ventaja medible frente a los clásicos en problemas de ciencia de datos. Para responderla se compararon seis modelos de clasificación sobre Iris, Bars and Stripes e Ising y seis solvers de optimización sobre TSP, MDS y MaxCut, con un protocolo común y la representación como eje de lectura. Las conclusiones siguen de los resultados del Capítulo 5.

6.1 Conclusiones

1. **En clasificación no se observa ventaja cuántica.** El SVM con kernel RBF obtiene el mejor F1 o empata con el mejor en los tres datasets (0.967 en Iris, 0.870 en Bars and Stripes, 0.985 en Ising) y es el más rápido de entrenar. Ningún modelo cuántico lo supera. La media cuántica de accuracy queda 0.083 y 0.101 por debajo de la clásica en Iris y Bars and Stripes, y la iguala en Ising.

2. **El QSVM con mapa ZZ solo alcanza al clásico en el dataset más fácil.** Empata con el SVM en Ising (F1 0.985), donde los seis modelos alcanzan al menos 0.95 de accuracy, y cae a 0.730 en Iris y 0.758 en Bars and Stripes, con el mayor gap de generalización del corpus en Iris (0.233). El experimento no aísla la causa del empate en Ising: el mapa actúa sobre componentes principales de las amplitudes, no sobre los espines de la cadena.

3. **Los modelos variacionales se acercan al clásico sin superarlo.** La QNN híbrida es el modelo cuántico más regular (F1 0.933, 0.815 y 0.937, con gap de generalización ≤ 0.022). El VQC iguala a los árboles en Iris (0.950), queda el último en Bars and Stripes (0.688) y es el más caro de entrenar en los tres datasets.

4. **En optimización, QAOA y VQE reproducen el óptimo, pero no mejoran a los métodos clásicos.** Los métodos exactos alcanzan $C^\star$ en las tres instancias, la enumeración en menos de un milisegundo. El annealing simulado lo hace sobre la misma formulación QUBO en milisegundos (éxito 0.9 en TSP y 1 en MDS y MaxCut). QAOA y VQE llegan al óptimo en TSP y MaxCut, y VQE también en MDS, con entre 0.6 y 2 s de simulación. En MDS una de las ocho semillas de QAOA devuelve una solución infactible (éxito 0.875, 12.6 s): termina con mayor energía

esperada y probabilidad nula de medir el óptimo, mientras que el annealing sobre el mismo QUBO acierta siempre. Como en TSP y MDS ambos leen el mejor de miles de bitstrings, su éxito en esos casos no prueba que el circuito concentre probabilidad en el óptimo.

5. **El régimen estudiado mide proximidad al clásico, no ventaja.** Con 4 qubits en clasificación, como mucho 6 en optimización y simulación de estado exacta, los resultados no permiten afirmar ni descartar una ventaja a escala (Preskill, 2018; Thanasilp et al., 2024). Sí permiten afirmar que, en este rango, ningún modelo o solver cuántico supera al mejor clásico, y que la paridad de calidad, cuando aparece, va acompañada de un coste temporal mayor.

7. Líneas futuras de investigación

La Tabla 7.1 recoge las líneas que extienden el estudio. Todas mantienen el criterio de comparación: el modelo cuántico frente al mejor clásico, con el mismo protocolo.

Línea	**Qué se propone**
Más qubits en hardware cuántico	Ejecutar los casos con más qubits en hardware real, por shots y con mitigación de errores, con instancias mayores de HamLib (Sawaya et al., 2024; PennyLane, n.d.g), kernels estimados por medidas aleatorizadas (Haug et al., 2023) y compilación estructurada (Rovara et al., 2026).
Mapeos de representación	Comparar otros feature maps y codificaciones: basis encoding, re-uploading (Pérez-Salinas et al., 2020), embeddings entrenables (Lloyd et al., 2020) y elección del circuito según la complejidad del dato (de Paula Neto, 2025).
Más familias de casos de uso	Extender el protocolo a clusterización (Aïmeur et al., 2007; Sentís et al., 2019; Kerenidis et al., 2019), series de tiempo, regresión (Friedman, 1991; Kulkarni, n.d.) y simulación.
Reducción de dimensionalidad	Sustituir la PCA por otros métodos, clásicos y cuánticos, como qPCA o autoencoders cuánticos (Lloyd et al., 2014; Romero et al., 2017), con la salvedad de Tang (2021).
Instancias diferentes a QUBO	Problemas cuya formulación natural no es cuadrática, como Max-k-SAT, y otras familias de Hamiltonianos de HamLib (Sawaya et al., 2024).
Diferentes conjuntos de datos	Repetir la comparación sobre otros datasets, reales y sintéticos, como los del benchmark de Bowles et al. (2024).

Cuadro 7.1: Líneas futuras de investigación.

Fuente: elaboración propia.

Bibliografía

Aïmeur, E., Brassard, G., & Gambs, S. (2007). *Quantum clustering algorithms*. Proceedings of the 24th International Conference on Machine Learning (ICML '07), 1–8. https://doi.org/10.1145/1273496.1273497

Arute, F., Arya, K., Babbush, R., Bacon, D., Bardin, J. C., Barends, R., ... Martinis, J. M. (2019). *Quantum supremacy using a programmable superconducting processor*. Nature, 574(7779), 505–510. https://doi.org/10.1038/s41586-019-1666-5

Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., & Lloyd, S. (2017). *Quantum machine learning*. Nature, 549(7671), 195–202. https://doi.org/10.1038/nature23474

Bowles, J., Ahmed, S., & Schuld, M. (2024). *Better than classical? The subtle art of benchmarking quantum machine learning models*. arXiv preprint arXiv:2403.07059. https://doi.org/10.48550/arXiv.2403.07059

Breiman, L. (2001). *Random forests*. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Cerezo, M., Arrasmith, A., Babbush, R., Benjamin, S. C., Endo, S., Fujii, K., ... Coles, P. J. (2021). *Variational quantum algorithms*. Nature Reviews Physics, 3(9), 625–644. https://doi.org/10.1038/s42254-021-00348-9

Cerezo, M., Larocca, M., García-Martín, D., Diaz, N. L., Braccia, P., Fontana, E., Rudolph, M. S., Bermejo, P., Ijaz, A., Thanasilp, S., Anschuetz, E. R., & Holmes, Z. (2025). *Does provable absence of barren plateaus imply classical simulability?* Nature Communications, 16(1), 7907. https://doi.org/10.1038/s41467-025-63099-6

CERN & Atos. (2021). *CERN–Atos Quantum Computing Fast Start* (session materials). https://indico.cern.ch/event/1098063/attachments/2357402/4023253/CERN_Atos_Fast-Start_3.pdf

Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

Cortes, C., Mohri, M., & Rostamizadeh, A. (2012). *Algorithms for learning kernels based on centered alignment*. Journal of Machine Learning Research, 13, 795–828.

Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. Machine Learning, 20(3), 273–297. https://doi.org/10.1007/BF00994018

Deutsch, D. (1985). *Quantum theory, the Church–Turing principle and the universal quantum computer*. Proceedings of the Royal Society of London A, 400(1818), 97–117. https://doi.org/10.1098/rspa.1985.0070

Farhi, E., Goldstone, J., & Gutmann, S. (2014). *A quantum approximate optimization algorithm*. arXiv preprint arXiv:1411.4028. https://doi.org/10.48550/arXiv.1411.4028

Fastovets, D. V., Bogdanov, Yu. I., Bantysh, B. I., & Lukichev, V. F. (2019). *Machine learning methods in quantum computing theory*. arXiv preprint arXiv:1906.10175. https://doi.org/10.48550/arXiv.1906.10175

Feynman, R. P. (1982). *Simulating physics with computers*. International Journal of Theoretical Physics, 21(6–7), 467–488. https://doi.org/10.1007/BF02650179

Friedman, J. H. (1991). *Multivariate adaptive regression splines*. The Annals of Statistics, 19(1), 1–67. https://doi.org/10.1214/aos/1176347963

Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman and Company.

Google Quantum AI and Collaborators. (2025). *Quantum error correction below the surface code threshold*. Nature, 638(8052), 920–926. https://doi.org/10.1038/s41586-024-08449-y

Grover, L. K. (1996). *A fast quantum mechanical algorithm for database search*. Proceedings of the 28th Annual ACM Symposium on Theory of Computing, 212–219. https://doi.org/10.1145/237814.237866

Gurobi Optimization. (n.d.). *Integer linear programming* (FAQ). https://www.gurobi.com/resources/faq/integer-linear-programming

Haug, T., Self, C. N., & Kim, M. S. (2023). *Quantum machine learning of large datasets using randomized measurements*. Machine Learning: Science and Technology, 4(1), 015005. https://doi.org/10.1088/2632-2153/acb0b4

Havlíček, V., Córcoles, A. D., Temme, K., Harrow, A. W., Kandala, A., Chow, J. M., & Gambetta, J. M. (2019). *Supervised learning with quantum-enhanced feature spaces*. Nature, 567(7747), 209–212. https://doi.org/10.1038/s41586-019-0980-2

Huang, H.-Y., Broughton, M., Mohseni, M., Babbush, R., Boixo, S., Neven, H., & McClean, J. R. (2021). *Power of data in quantum machine learning*. Nature Communications, 12(1), 2631. https://doi.org/10.1038/s41467-021-22539-9

Jordan, M. I., & Mitchell, T. M. (2015). *Machine learning: Trends, perspectives, and prospects*. Science, 349(6245), 255–260. https://doi.org/10.1126/science.aaa8415

Kerenidis, I., Landman, J., Luongo, A., & Prakash, A. (2019). *q-means: A quantum algorithm for unsupervised machine learning*. Advances in Neural Information Processing Systems, 32. https://doi.org/10.48550/arXiv.1812.03584

Kulkarni, A. (n.d.). *Quantum-Linear-Regression* (software repository). GitHub. https://github.com/amey-kulkarni27/Quantum-Linear-Regression

Liu, Y., Arunachalam, S., & Temme, K. (2021). *A rigorous and robust quantum speed-up in supervised machine learning*. Nature Physics, 17(9), 1013–1017. https://doi.org/10.1038/s41567-021-01287-z

Lloyd, S., Mohseni, M., & Rebentrost, P. (2014). *Quantum principal component analysis*. Nature Physics, 10(9), 631–633. https://doi.org/10.1038/nphys3029

Lloyd, S., Schuld, M., Ijaz, A., Izaac, J., & Killoran, N. (2020). *Quantum embeddings for machine learning*. arXiv preprint arXiv:2001.03622. https://doi.org/10.48550/arXiv.2001.03622

Lubinski, T., Coffrin, C., McGeoch, C., Sathe, P., Apanavicius, J., Bernal Neira, D., & QED-C Collaboration. (2024). *Optimization applications as quantum performance benchmarks*. ACM Transactions on Quantum Computing, 5(3). https://doi.org/10.1145/3678184

Lucas, A. (2014). *Ising formulations of many NP problems*. Frontiers in Physics, 2, 5. https://doi.org/10.3389/fphy.2014.00005

McClean, J. R., Boixo, S., Smelyanskiy, V. N., Babbush, R., & Neven, H. (2018). *Barren plateaus in quantum neural network training landscapes*. Nature Communications, 9(1), 4812. https://doi.org/10.1038/s41467-018-07090-4

Nielsen, M. A., & Chuang, I. L. (2010). *Quantum computation and quantum information* (10th anniversary ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511976667

Pan, F., Chen, K., & Zhang, P. (2022). *Solving the sampling problem of the Sycamore quantum circuits*. Physical Review Letters, 129(9), 090502. https://doi.org/10.1103/PhysRevLett.129.090502

Parate, A. (2025). *A complete guide to quantum data encoding*. Medium. https://medium.com/@parateaadish/a-complete-guide-to-quantum-data-encoding-786554b346ab

Parzygnat, A. J., Bradley, T.-D., Vlasic, A., & Pham, A. (2025). *Toward structure-preserving quantum encodings*. Physical Review Research. https://doi.org/10.1103/rph8-g15q

de Paula Neto, F. M. (2025). *Data complexity measures for quantum circuits architecture recommendation*. arXiv preprint arXiv:2502.15129. https://doi.org/10.48550/arXiv.2502.15129

PennyLane / Xanadu. (n.d.). *Bars and Stripes* (dataset). https://pennylane.ai/datasets/bars-and-stripes

PennyLane / Xanadu. (n.d.). *Conventional datasets for QML*. https://pennylane.ai/datasets/conventional-datasets-for-qml

Schoch, I. / PennyLane. (2022). *How to embed data into a quantum state*. PennyLane Blog. https://pennylane.ai/blog/2022/08/how-to-embed-data-into-a-quantum-state/

PennyLane / Xanadu. (n.d.). *Quantum embedding* (glossary). https://pennylane.ai/glossary/quantum_embedding

PennyLane / Xanadu. (n.d.). *Grover's algorithm* (demo). https://pennylane.ai/demos/tutorial_grovers_algorithm

PennyLane / Xanadu. (n.d.). *HamLib MaxCut* (dataset). https://pennylane.ai/datasets/hamlib-maxcut

PennyLane / Xanadu. (n.d.). *HamLib travelling salesperson problem* (dataset). https://pennylane.ai/datasets/hamlib-travelling-salesperson-problem

PennyLane / Xanadu. (n.d.). *Ising* (dataset / qspin). https://pennylane.ai/datasets/ising

PennyLane / Xanadu. (n.d.). *Intro to QAOA* (demo). https://pennylane.ai/demos/tutorial_qaoa_intro

Lowe, A., & Soni, J. / PennyLane. (2019). *QAOA for MaxCut* (demo). https://pennylane.ai/demos/tutorial_qaoa_maxcut

PennyLane / Xanadu. (n.d.). *QFT and classical groups* (demo). https://pennylane.ai/demos/tutorial_qft_and_groups

PennyLane / Xanadu. (n.d.). *Quantum transfer learning* (demo). https://pennylane.ai/demos/tutorial_quantum_transfer_learning

PennyLane / Xanadu. (n.d.). *Variational classifier* (demo). https://pennylane.ai/demos/tutorial_variational_classifier

PennyLane / Xanadu. (n.d.). *What is a quantum dataset?* https://pennylane.ai/datasets/what-is-a-quantum-dataset

Pérez-Salinas, A., Cervera-Lierta, A., Gil-Fuster, E., & Latorre, J. I. (2020). *Data re-uploading for a universal quantum classifier*. Quantum, 4, 226. https://doi.org/10.22331/q-2020-02-06-226

Peruzzo, A., McClean, J., Shadbolt, P., Yung, M.-H., Zhou, X.-Q., Love, P. J., ... O'Brien, J. L. (2014). *A variational eigenvalue solver on a photonic quantum processor*. Nature Communications, 5(1), 4213. https://doi.org/10.1038/ncomms5213

Preskill, J. (2018). *Quantum computing in the NISQ era and beyond*. Quantum, 2, 79. https://doi.org/10.22331/q-2018-08-06-79

Qiskit contributors. (2026). *Qiskit: An open-source framework for quantum computing* (Versions 2.3.0 and 2.6.1). IBM Quantum. https://doi.org/10.5281/zenodo.2573505

Quantum Zeitgeist. (n.d.). *PennyLane and quantum machine learning* (hybrid quantum-classical algorithms). https://quantumzeitgeist.com/pennylane-and-quantum-machine-learning/#hybrid-quantum-classical-algorithms

Quantum Zeitgeist. (n.d.). *Understanding quantum algorithms: A beginner's guide.* https://quantumzeitgeist.com/understanding-quantum-algorithms-a-beginners-guide/

Quantum Zeitgeist. (n.d.). *FBI battles foreign adversaries targeting U.S. quantum tech.* https://quantumzeitgeist.com/fbi-battles-foreign-adversaries-targeting-u-s-quantum-tech-bolstering-national-security/

Romero, J., Olson, J. P., & Aspuru-Guzik, A. (2017). *Quantum autoencoders for efficient compression of quantum data.* Quantum Science and Technology, 2(4), 045001. https://doi.org/10.1088/2058-9565/aa8072

Rovara, D., Haag, D., Koch, M., Izaac, J., Sivarajah, S., Wille, R., … Ittah, D. (2026). *Why are we unrolling? The importance of structured quantum programs for compilation.* arXiv preprint arXiv:2609.16171. https://doi.org/10.48550/arXiv.2609.16171

Sawaya, N. P. D., Marti-Dafcik, D., Ho, Y., Tabor, D. P., Bernal Neira, D. E., Magann, A. B., … Camps, D. (2024). *HamLib: A library of Hamiltonians for benchmarking quantum algorithms and hardware.* arXiv preprint arXiv:2306.13126. https://doi.org/10.48550/arXiv.2306.13126

Schuld, M. (2021). *Supervised quantum machine learning models are kernel methods.* arXiv preprint arXiv:2101.11020. https://doi.org/10.48550/arXiv.2101.11020

Schuld, M., & Killoran, N. (2019). *Quantum machine learning in feature Hilbert spaces.* Physical Review Letters, 122(4), 040504. https://doi.org/10.1103/PhysRevLett.122.040504

scikit-learn developers. (n.d.). *The Iris Dataset* (example). https://scikit-learn.org/1.5/auto_examples/datasets/plot_iris_dataset.html

Sentís, G., Monràs, A., Muñoz-Tapia, R., Calsamiglia, J., & Bagan, E. (2019). *Unsupervised classification of quantum data.* Physical Review X, 9(4), 041029. https://doi.org/10.1103/PhysRevX.9.041029

Shor, P. W. (1994). *Algorithms for quantum computation: Discrete logarithms and factoring.* Proceedings of the 35th Annual Symposium on Foundations of Computer Science, 124–134. https://doi.org/10.1109/SFCS.1994.365700

Sim, S., Johnson, P. D., & Aspuru-Guzik, A. (2019). *Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms.* Advanced Quantum Technologies, 2(12), 1900070. https://doi.org/10.1002/qute.201900070

Somma, R. D., Boixo, S., & Barnum, H. (2008). *Quantum simulated annealing.* arXiv preprint arXiv:0712.1008. https://doi.org/10.48550/arXiv.0712.1008

Tang, E. (2021). *Quantum principal component analysis only achieves an exponential speedup because of its state preparation assumptions.* Physical Review Letters, 127(6), 060503. https://doi.org/10.1103/PhysRevLett.127.060503

Thanasilp, S., Wang, S., Cerezo, M., & Holmes, Z. (2024). *Exponential concentration in quantum kernel methods.* Nature Communications, 15(1), 5200. https://doi.org/10.1038/s41467-024-49287-w

Vikstål, P., Grönkvist, M., Svensson, M., Andersson, M., Johansson, G., & Ferrini, G. (2020). *Applying the quantum approximate optimization algorithm to the tail-assignment problem.* Physical Review Applied, 14(3), 034009. https://doi.org/10.1103/PhysRevApplied.14.034009

Vocal Media. (n.d.). *Quantum algorithms outperform classical solvers in optimization tasks, study reveals.* https://vocal.media/education/quantum-algorithms-outperform-classical-solvers-in-optimization-tasks-study-reveals

Wang, G. (n.d.). *Classical versus quantum machine learning: A comparative study.* Medium. https://medium.com/@gwrx2005/classical-versus-quantum-machine-learning-a-comparative-study-1e65ced52a42

Wolpert, D. H., & Macready, W. G. (1997). *No free lunch theorems for optimization.* IEEE Transactions on Evolutionary Computation, 1(1), 67–82. https://doi.org/10.1109/4235.585893

XanaduAI. (n.d.). *qml-benchmarks* (software repository). GitHub. https://github.com/XanaduAI/qml-benchmarks

Yu, C., Liu, J., Ge, Z., Wu, X., Zhu, L., Zhao, P., & Zhao, J. (2026). *Quantum vs. classical machine learning: A unified empirical comparison.* arXiv preprint arXiv:2607.01197. https://doi.org/10.48550/arXiv.2607.01197

Z-537-437. (n.d.). *QML* (software repository accompanying Yu et al.). GitHub. https://github.com/Z-537-437/QML

Zhou, L., Wang, S.-T., Choi, S., Pichler, H., & Lukin, M. D. (2020). *Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices.* Physical Review X, 10(2), 021067. https://doi.org/10.1103/PhysRevX.10.021067

A. Apéndices

A.1 Repositorio del proyecto

El código, los datos de entrada, las métricas, las figuras, este documento y las diapositivas de la defensa están en un repositorio público de GitHub, con `main` como rama principal:

`https://github.com/juandapradam12/UNIR-MasterQC-QuantumAdvantageInDataScience`

La Tabla A.1 describe el primer nivel del repositorio.

Elemento	**Contenido**
`Notebooks/`	Corpus experimental: un notebook por modelo o solver, sus métricas en JSON y sus figuras
`Notebooks/lib/`	Utilidades comunes: resolución de las carpetas `metrics/` y `figures/` de cada caso (`case_paths.py`) y figuras de fronteras, instancias y soluciones (`storytelling.py`)
`Documento/`	Fuente LaTeX de esta memoria: `main.tex`, un fichero por capítulo en `Capitulos/`, estilo UNIR en `Estilos/` e imágenes en `Imagenes/`
`Defensa/`	Diapositivas de la defensa en Beamer (`defensa.tex`), que reutilizan las figuras de `Notebooks/` y `Documento/Imagenes/`
`Notas/`	Notas de trabajo y referencias candidatas
`requirements.txt`	Dependencias de Python con versión fijada

Cuadro A.1: Primer nivel del repositorio.

Fuente: elaboración propia.

La Figura A.1 muestra el árbol de carpetas. Las ocho carpetas de caso repiten la misma estructura, que se detalla para Iris y para TSP.

A.2 Catálogo de notebooks

La Tabla A.2 resume la organización del corpus experimental, repartido entre las carpetas `Notebooks/Classification/` y `Notebooks/Optimization/`. Cada dataset o instancia

```
UNIR-MasterQC-QuantumAdvantageInDataScience/
|-- requirements.txt
|-- Notebooks/
|   |-- lib/                      case_paths.py, storytelling.py
|   |-- Classification/
|   |   |-- IrisClassification/
|   |   |   |-- Classic_SVM_Iris.ipynb, RandomForest_Iris.ipynb,
|   |   |   |   XGBoost_Iris.ipynb, QSVM_Iris.ipynb, QNN_Iris.ipynb,
|   |   |   |   Variational_Classifier_Iris.ipynb
|   |   |   |-- metrics/          svm_iris.json, ..., vqc_iris.json
|   |   |   `-- figures/
|   |   |-- BSClassification/     misma estructura
|   |   |-- IsingClassification/  misma estructura
|   |   `-- ResultsClassification/
|   |       |-- Results_Iris.ipynb, Results_BS.ipynb, Results_Ising.ipynb,
|   |       |   Results_Classification.ipynb
|   |       |-- metrics/          resúmenes CSV y medias en JSON
|   |       `-- figures/
|   `-- Optimization/
|       |-- TSPOptimization/
|       |   |-- BruteForce.ipynb, Greedy.ipynb, ILP.ipynb,
|       |   |   QUBO_Simulated_Annealing.ipynb, QUBO_QAOA.ipynb,
|       |   |   QUBO_VQE.ipynb
|       |   |-- metrics/          bruteforce_tsp.json, ..., vqe_tsp.json
|       |   `-- figures/
|       |-- MinimumDominatingSetOptimization/   misma estructura
|       |-- MaxCutOptimization/   misma estructura, más
|       |   `-- instance/         hamlib_maxcut.json
|       `-- ResultsOptimization/
|           |-- Results_TSP.ipynb, Results_MDS.ipynb, Results_MaxCut.ipynb,
|           |   Results_Optimization.ipynb
|           |-- metrics/          resúmenes CSV
|           `-- figures/
|-- Documento/
|   |-- main.tex
|   |-- Capitulos/  Estilos/  Imagenes/
|-- Defensa/
|   `-- defensa.tex
`-- Notas/
```

Figura A.1: Árbol de carpetas del repositorio. Se omiten las figuras y los PDF compilados.
Fuente: elaboración propia.

tiene un notebook por modelo o solver, y una carpeta de resultados agrega las métricas y genera las figuras comparativas del Capítulo 5.

Carpeta	**Contenido**	**Datos**
`IrisClassification/`	SVM, RF, XGBoost, QSVM, QNN, VQC	Iris
`BSClassification/`	mismos seis modelos	Bars and Stripes
`IsingClassification/`	mismos seis modelos	Ising 1D
`ResultsClassification/`	agregación de métricas y figuras comparativas	tres datasets
`TSPOptimization/`	fuerza bruta, greedy, ILP, QUBO+SA, QAOA, VQE	TSP
`MinimumDominating-SetOptimization/`	mismos seis solvers	MDS
`MaxCutOptimization/`	mismos seis solvers	MaxCut (HamLib)
`ResultsOptimization/`	agregación de métricas y figuras comparativas	tres instancias

Cuadro A.2: Organización del corpus experimental.

Fuente: elaboración propia.

A.3 Formato de las salidas

Cada notebook de modelo o solver escribe un JSON en la carpeta `metrics/` de su caso. Los notebooks de resultados solo leen esos JSON y escriben los resúmenes CSV y las figuras comparativas (Tabla A.3).

A.4 Reproducción

Los notebooks localizan `Notebooks/lib/` y las carpetas de su caso subiendo por los directorios padre, así que pueden ejecutarse desde la raíz del repositorio o desde su propia carpeta. La Tabla A.4 recoge el orden de ejecución.

Fichero	**Campos**
JSON de clasificación	`model`, `family`, `protocol`; `train` y `test` con accuracy, precision, recall, F1 y AUC; `train_time_s`, `inference_time_s`
JSON de optimización	`problem`, `solver`, `family`, `protocol`, `instance`; `quality` (óptimo, coste medio y mejor, gap medio), `success_rate`, `time_s`, `effort`; `runs` con una entrada por semilla (bitstring, solución, coste, factibilidad, éxito, gap, tiempo y, en QAOA y VQE, evaluaciones de energía y profundidad del circuito)
Instancia MaxCut	`hamlib_maxcut.json`: origen e identificador en HamLib, número de vértices, aristas, corte óptimo y bitstrings óptimos
Resúmenes CSV	Una tabla por dataset o instancia y tablas agregadas de clasificación y de optimización, en las carpetas `metrics/` de resultados

Cuadro A.3: Ficheros de salida del corpus experimental.

Fuente: elaboración propia.

Paso	**Acción**	**Detalle**
1	Entorno	Entorno virtual con Python 3.14; `pip install -r requirements.txt`; `qiskit-machine-learning` se instala aparte porque no está fijado
2	Notebooks de caso	Ejecutar los notebooks de modelo o solver de cada carpeta de caso, en cualquier orden; cada uno escribe su JSON y sus figuras
3	Notebooks de resultados	Ejecutar `Results_*.ipynb` en `ResultsClassification/` y `ResultsOptimization/`, que regeneran los CSV y las figuras comparativas
4	Documento y defensa	Compilar `Documento/main.tex` y `Defensa/defensa.tex` con `pdflatex`, dos pasadas cada uno

Cuadro A.4: Pasos para reproducir los resultados, el documento y la defensa.

Fuente: elaboración propia.